

INDEX

TOPICS	Page No's
➤ Certificates	
➤ Acknowledgement	
➤ Abstract	
➤ Figures/Tables	
CHAPTER-1: INTRODUCTION	1-2
CHAPTER-2: LITERATURE SURVEY	3-5
CHAPTER-3: SYSTEM ANALYSIS	
3.1 Existing System	6
3.2 Proposed System	7
CHAPTER-4: SYSTEM REQUIREMENTS	
4.1 Hardware Requirements	8
4.2 Software Requirements	8
CHAPTER-5: SYSTEM STUDY	
5.1 Feasibility Study	9
5.2 Feasibility Analysis	9-10
CHAPTER-6: SYSTEM ARCHITECTURE	
6.1 SYSTEM ARCHITECTURE	11
6.2 UML Diagrams	12-20
6.2.1 Use Case Diagram	
6.2.2 Class Diagram	
6.2.3 Sequence Diagram	
6.2.4 Collaboration Diagram	
6.2.5 Activity Diagram	

6.2.6 Component Diagram

6.2.7 Deployment Diagram

6.2.8 Er Diagram

6.2.9 Data Set

CHAPTER-7: INPUT AND OUTPUT DESIGN

7.1 Input Design 21

7.2 Output Design 22

CHAPTER-8: IMPLEMENTATION

8.1 MODULES 23

8.1.1 Module Description 23

CHAPTER-9: SOFTWARE ENVIRONMENT

9.1 Python 24-63

9.2 Source Code 63-72

CHAPTER-10: RESULTS/DISCUSSIONS

10.1 System Test 73-76

10.1.1 Test Cases 76-78

10.2 Screenshots 79-87

CHAPTER-11: CONCLUSION

11.1 Conclusion 88

11.2 Future Scope 88-89

CHAPTER-12: REFERENCES/BIBLIOGRAPHY 90-91

LIST OF FIGURES

S.NO	TABLES/FIGURES	PAGE NO'S
1	System Architecture	11
2	UML Diagrams	12-20
	2.1 Use Case Diagram	13
	2.2 Class Diagram	13
	2.3 Sequence Diagram	14
	2.4 Collaboration Diagram	15
	2.5 Activity Diagram	16
	2.6 Component Diagram	17
	2.7 Deployment Diagram	17
	2.8 ER Diagram	18
	2.9 Data Dictionary/Data Set	9-201
3	About Python	24-63
4	Screenshots	79-87

AI POWERED SERVER LOG MANAGEMENT SOFTWARE

ABSTRACT

The rapid evolution of technology has led to an increase in errors, creating a challenge for users who face the daunting task of finding solutions. Traditionally, users navigate through various servers, submitting queries to seek solutions, only to receive a multitude of responses. Identifying the correct solution among these disparate server responses proves to be a time-consuming and challenging endeavor for users.

To alleviate this issue and streamline the process, we have implemented a solution that involves aggregating error logs from all servers. By employing Artificial Intelligence algorithms, we train a sophisticated AI model with the collected error questions and their corresponding solutions. This trained AI model is capable of ingesting a user's test error as input and predicting a possible close solution to the given error. This innovative approach transforms the user experience, eliminating the need to sift through multiple servers for answers and significantly reducing the time and burden associated with error resolution.

With this AI-driven model, users can obtain solutions from a single, centralized server, providing a one-stop solution for error resolution. While online search engines like BING and Google may offer solutions, their results may lack accuracy, and the abundance of responses from Google can be overwhelming. To ensure the effectiveness of our AI model, we have curated a dataset containing relevant questions and answers related to operating systems and programming. Users can also contribute to expanding the dataset by adding new questions and answers, facilitating the training of the AI model across various domains such as OS errors, programming errors, and beyond. This holistic approach ensures that users receive accurate and relevant solutions to their errors from a reliable and efficient source.

CHAPTER-1

INTRODUCTION

A. INTRODUCTION

In the ever-evolving landscape of technology, where errors and issues are inherent, managing server logs and providing efficient solutions has become a critical challenge. Traditional methods involve users navigating through multiple servers, searching for solutions to errors, resulting in time-consuming and often cumbersome experiences. To address this, our project focuses on the development of an AI-powered Server Log Management Software. This innovative solution aims to streamline the process of error resolution by harnessing the power of Artificial Intelligence to interpret server logs and provide users with accurate and timely solutions.

In today's digital landscape, organizations generate vast amounts of data through their server logs. These logs are critical for maintaining operational efficiency, ensuring security, and achieving regulatory compliance. However, the sheer volume and complexity of log data can be overwhelming. Traditional log management systems often fall short in providing timely and actionable insights. This is where AI-powered server log management software comes into play, offering a transformative solution to handle, analyze, and derive value from log data.

THE NEED FOR ADVANCED LOG MANAGEMENT

Server logs are detailed records of activities within an IT environment, encompassing everything from user interactions and system processes to security events and error messages. Effective log management is essential for:

Operational Monitoring: Ensuring that systems are running smoothly and efficiently.

Security and Incident Response: Detecting and responding to security threats and breaches.

Compliance: Meeting industry regulations and standards by maintaining proper logs.

Troubleshooting and Diagnostics: Identifying and resolving issues quickly to minimize downtime.

Traditional log management tools often rely on basic storage, search, and manual analysis, which can be labor-intensive and prone to human error. With the exponential growth in data volume and the increasing sophistication of cyber threats, there is a critical need for more advanced solutions.

INTRODUCTION TO AI-POWERED LOG MANAGEMENT

AI-powered server log management software leverages artificial intelligence and machine learning to enhance the capabilities of traditional log management. By automating the analysis and interpretation of log data, these advanced systems can provide deeper insights, faster detection of anomalies, and more accurate predictions.

AI-powered server log management software represents a significant advancement in how organizations handle and derive value from their log data. By automating analysis, enhancing security, and providing predictive insights, these systems empower IT teams to be more proactive and efficient. As the volume and complexity of log data continue to grow, the adoption of AI-driven log management solutions will be crucial for maintaining robust, secure, and compliant IT environments.

CHAPTER-2

LITERATURE SURVEY

TITLE: AI Powered Server Log Management Software

AUTHOR: Shaik salma begum, amirisetty deepak venkat, s pranay kiran, akshay jayesh, kesava m

ABSTRACT: In software engineering, logging traditionally aids in root cause analysis and failure identification. Recent advancements in automated log file analysis have expanded its utility to include real-time system health monitoring, user behavior analysis, and domain knowledge extraction. However, the lack of a standardized format remains a significant obstacle. Focusing on the last five years, this paper summarizes key research topics, categorizes log files used in studies, and offers an informative foundation for future investigators, providing insights into diverse log data applications. Keywords: Artificial intelligence, Server log management, Error detection, Log analysis, Automated analysis, Log file standardization, Django framework, User behavior.

TITLE: DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning

AUTHOR: Min Du, Feifei Li, Guineng Zheng, and others

ABSTRACT: System logs are essential for diagnosing and understanding system behaviors, especially in complex IT environments. Traditional methods of log analysis are often manual and time-consuming, failing to scale with the growing volume and complexity of log data. This paper introduces DeepLog, an innovative framework for anomaly detection and diagnosis from system logs using deep learning techniques. DeepLog leverages Long Short-Term Memory (LSTM) networks to model the sequential patterns inherent in log data, learning the normal patterns of system behavior over time.

The framework operates in three primary phases: log parsing, anomaly detection, and diagnosis. In the log parsing phase, raw log data is structured into a format suitable for machine learning models. During the anomaly detection phase, the LSTM-based model predicts the next log event based on the learned patterns. Anomalies are detected when there is a significant deviation between the predicted and actual log events. For diagnosis, DeepLog provides interpretable results, helping system administrators identify the root cause of anomalies efficiently.

DeepLog was evaluated using real-world datasets from diverse system environments. The results demonstrate that DeepLog can accurately detect anomalies with high precision and recall, outperforming traditional methods. Moreover, the framework's ability to adapt to different systems without extensive manual tuning highlights its robustness and scalability. DeepLog represents a significant advancement in automated log analysis, offering a practical solution for proactive system monitoring and maintenance.

TITLE: LogLens: A Real-time Log Analysis System

AUTHOR: Yuting Zhang, Yongle Zhang, Dingding Li

ABSTRACT: In the realm of modern IT infrastructure, log data serves as a critical source for monitoring system performance, diagnosing issues, and ensuring security. The exponential growth of log data, driven by the proliferation of cloud computing, microservices, and IoT devices, necessitates advanced solutions for efficient and real-time log analysis. This paper presents LogLens, an AI-powered real-time log analysis system designed to address the challenges of large-scale log data processing and anomaly detection.

LogLens leverages a combination of machine learning techniques and advanced data processing architectures to provide a robust solution for log analysis. The system is composed of several key components: a log parsing module that structures unstructured log data, an anomaly detection engine that employs machine learning models to identify unusual patterns, and a real-time processing pipeline that ensures low-latency analysis and high throughput.

The architecture of LogLens is designed to handle the diverse and voluminous nature of log data. The log parsing module uses state-of-the-art natural language processing techniques to convert raw logs into structured formats, enabling easier analysis. The anomaly detection engine incorporates both supervised and unsupervised learning models, allowing it to detect known issues as well as uncover new, previously unseen anomalies. To support real-time requirements, LogLens utilizes a scalable data pipeline built on distributed computing frameworks, ensuring that the system can process and analyze logs with minimal delay.

TITLE: Automated Log-based Anomaly Detection for Cloud Computing Infrastructures

AUTHOR: Qiang Fu, Jie Xu

ABSTRACT: In the dynamic and complex environment of cloud computing, ensuring the reliability and security of services is paramount. Automated log-based anomaly detection has emerged as a critical tool in achieving these goals. This paper presents a comprehensive approach to leveraging artificial

intelligence (AI) techniques for detecting anomalies in cloud computing infrastructures through log analysis.

We explore the challenges posed by the massive volume and high velocity of log data generated in cloud environments. Our proposed solution employs advanced machine learning algorithms to parse, filter, and analyze log entries in real-time, facilitating the prompt identification of performance issues, security breaches, and other anomalies. The system architecture integrates unsupervised learning methods, such as clustering and anomaly detection models, to discern normal patterns from irregular behaviors without requiring extensive labeled datasets.

The effectiveness of our approach is demonstrated through extensive experiments on real-world cloud datasets. We provide a comparative analysis of different AI models, highlighting their accuracy, scalability, and efficiency in various cloud scenarios. The results indicate that our automated log-based anomaly detection system significantly reduces the time and effort required for manual log analysis, enhances the precision of anomaly detection, and supports proactive maintenance and security measures.

CHAPTER-3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

The existing system relies on users manually searching for solutions to errors across different servers. This process is not only time-consuming but also demands a considerable level of expertise to identify the correct solution from a multitude of server responses. The current approach lacks automation, leading to inefficiencies and a heightened burden on users seeking error resolution.

DISADVANTAGES

- **High False Positive Rates:** One of the primary challenges faced by automated log-based anomaly detection systems is the high false positive rate. Due to the complexity of cloud environments and the variability of normal behavior, these systems often generate alerts for events that are not actual anomalies, leading to alert fatigue and reduced effectiveness.
- **Limited Context Awareness:** Existing systems may lack contextual understanding of the logs they analyze. They may not consider the relationships between different log entries or the broader context of the cloud environment, resulting in missed anomalies or inaccurate assessments of severity.

3.2 PROPOSED SYSTEM

Our proposed solution involves aggregating error logs from various servers and training an Artificial Intelligence algorithm to understand and predict solutions to common errors. By utilizing machine learning techniques, the AI model becomes adept at comprehending user queries and providing close solutions to identified errors. This eliminates the need for users to sift through multiple servers, offering a one-stop, AI-powered server log management solution.

The system not only simplifies the error resolution process but also reduces the overall time and effort invested by users. Through continuous learning and refinement, the AI model ensures accuracy in predicting solutions across various domains, making it a versatile and reliable tool for server log management.

Advantages

- **Early Detection of Anomalies:** The automated system enables early detection of anomalies in cloud computing infrastructures, allowing for timely intervention before they escalate into major incidents.
- **Scalability and Efficiency:** By leveraging machine learning and real-time processing frameworks, the system can efficiently handle large volumes of log data generated by diverse cloud resources, ensuring scalability and responsiveness.
- **Reduced False Positives:** Advanced anomaly detection algorithms help minimize false positives by distinguishing between genuine anomalies and normal fluctuations in system behavior, reducing alert fatigue for administrators.

CHAPTER-4

SYSTEM REQUIREMENTS

4.1 HARDWARE REQUIREMENTS

- Processor - Pentium –IV
- RAM - 4 GB (min)
- Hard Disk - 20 GB
- Key Board - Standard Windows Keyboard
- Mouse - Two or Three Button Mouse
- Monitor - SVGA

4.2 SOFTWARE REQUIREMENTS

- ❖ Operating system : Windows 7 Ultimate.
- ❖ Coding Language : Python.
- ❖ Front-End : Python.
- ❖ Back-End : Django-ORM
- ❖ Designing : Html, css, javascript.
- ❖ Data Base : MySQL (WAMP Server).

CHAPTER-5

SYSTEM STUDY

5.1 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

5.2 FEASIBILITY ANALYSIS

Three key considerations involved in the feasibility analysis are

- ◆ ECONOMICAL FEASIBILITY
- ◆ TECHNICAL FEASIBILITY
- ◆ SOCIAL FEASIBILITY

ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will

lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

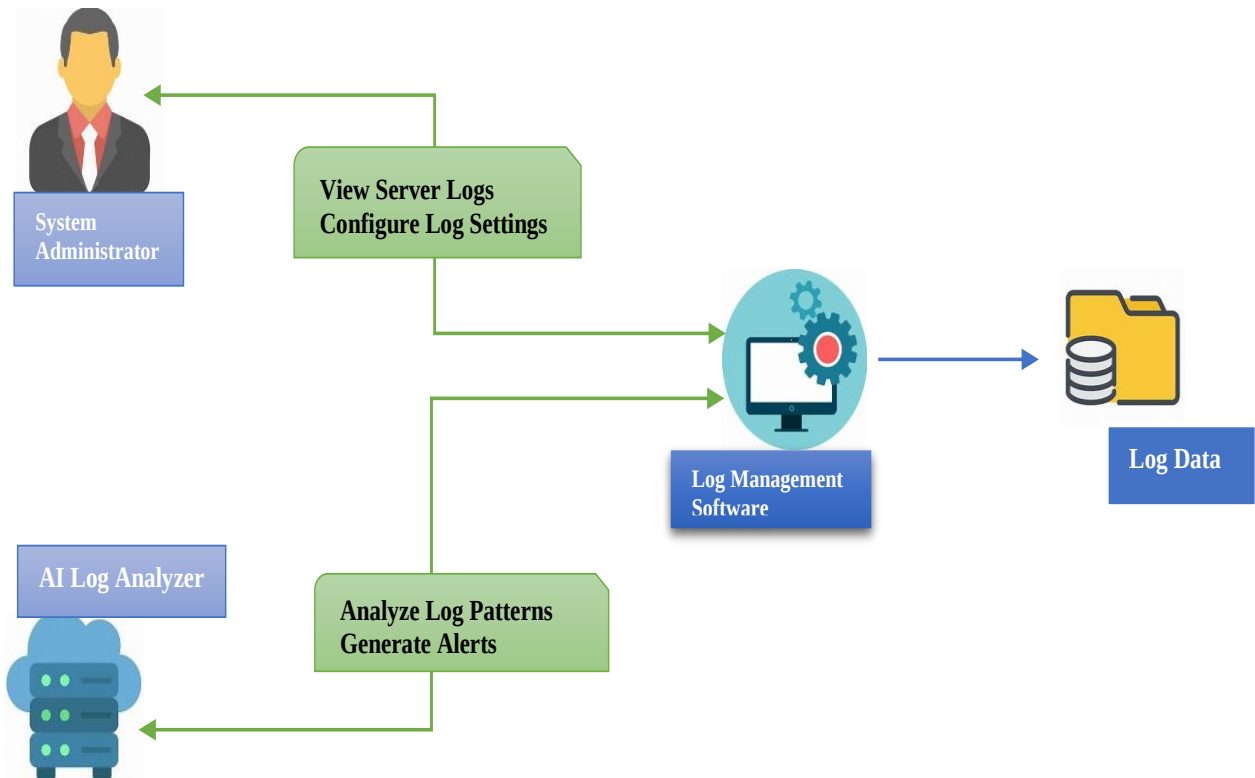
SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

CHAPTER-6

SYSTEM DESIGN

6.1 SYSTEM ARCHITECTURE



6.2 UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

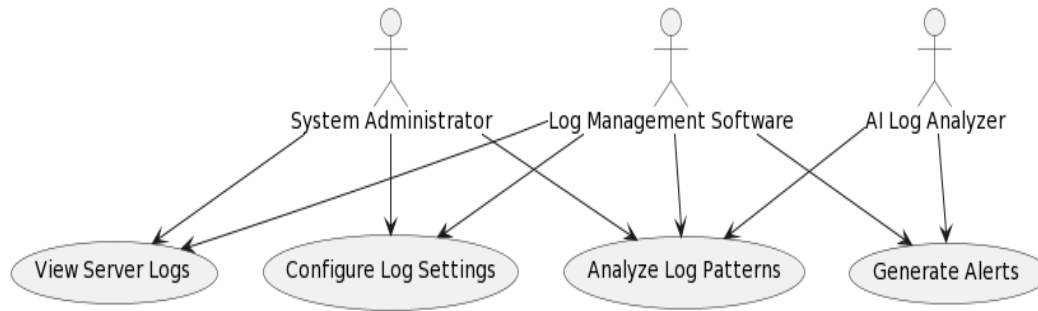
The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS

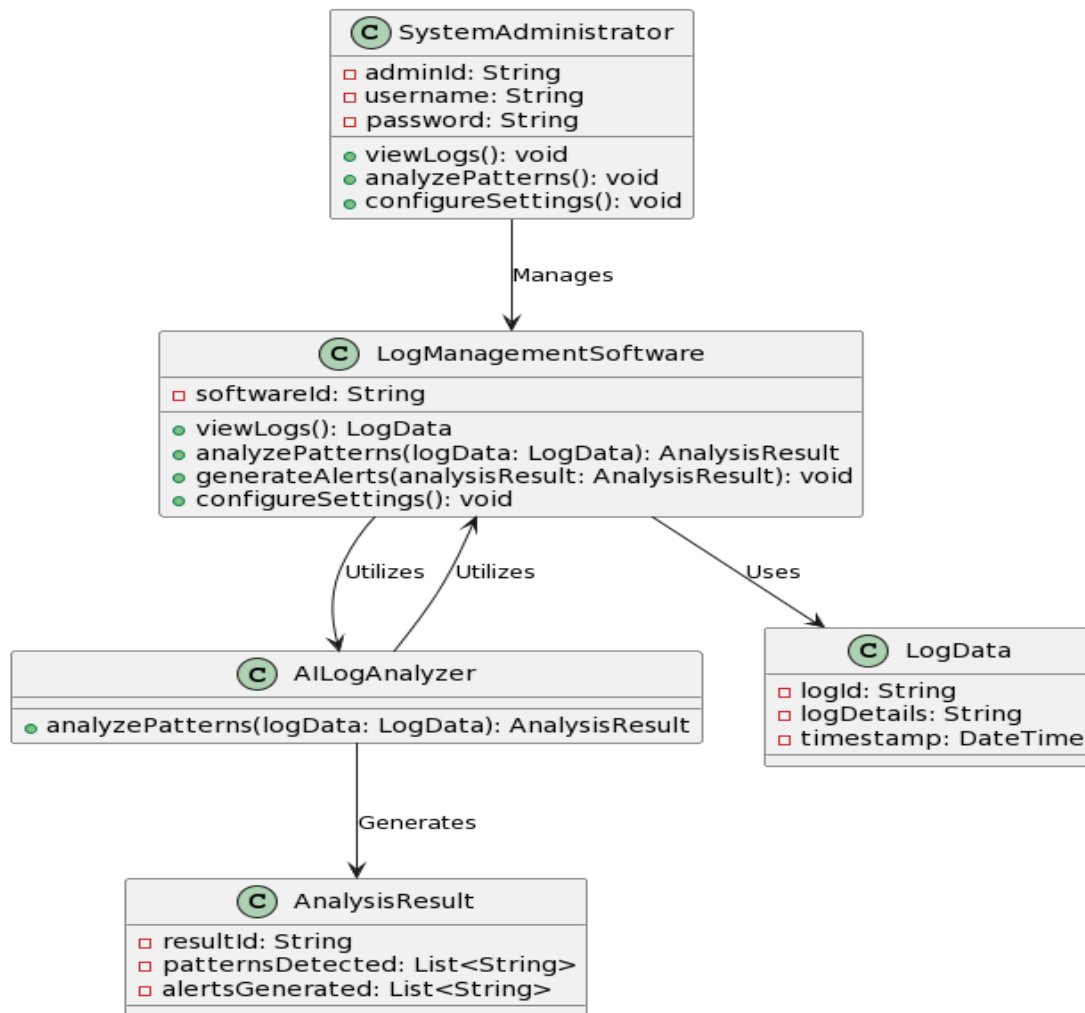
The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.

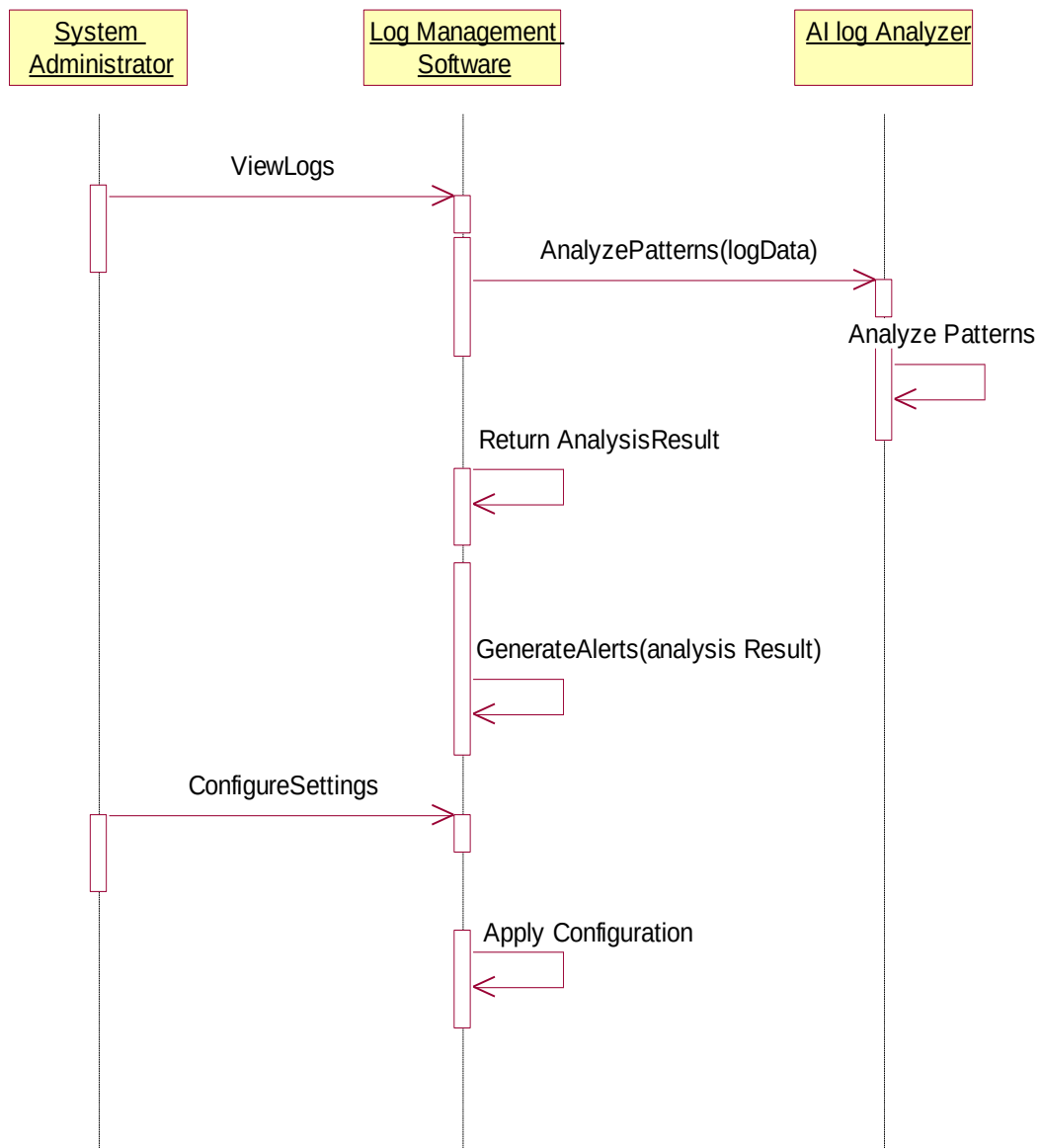
6.2.1 USECASE DIAGRAM



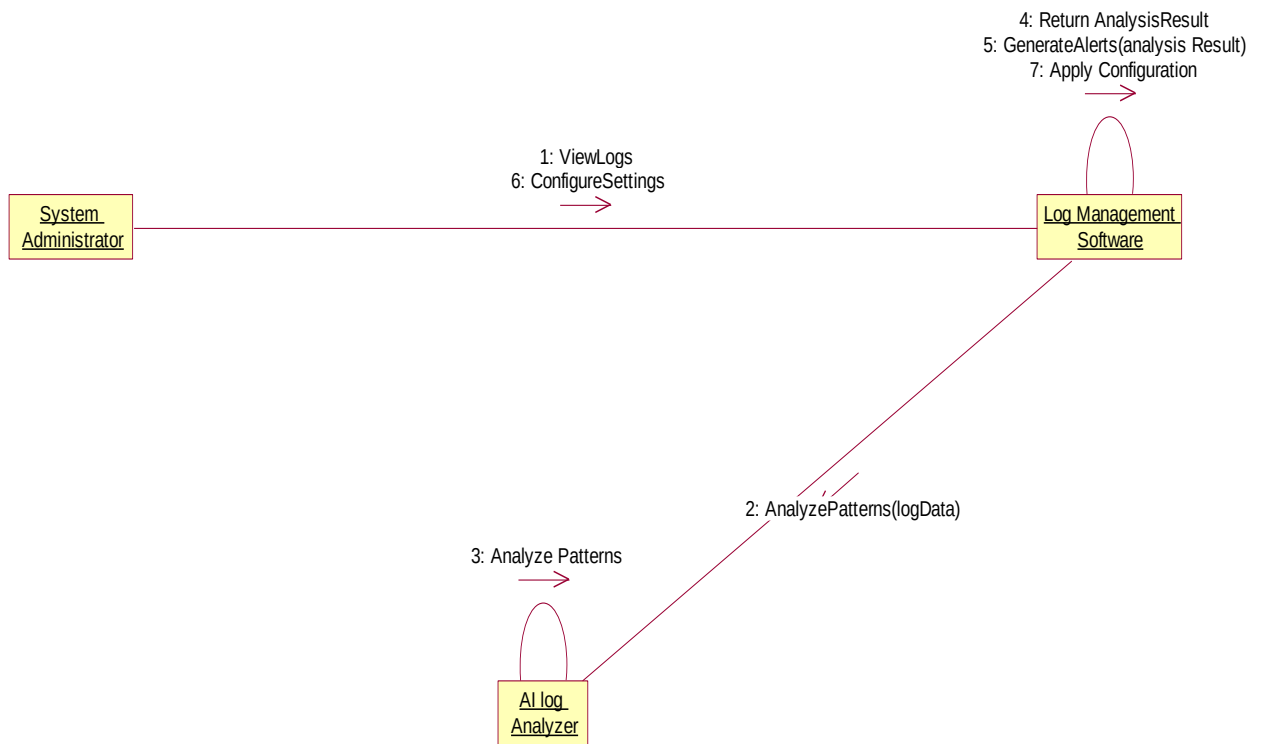
6.2.2 CLASS DIAGRAM



6.2.3 SEQUENCE DIAGRAM

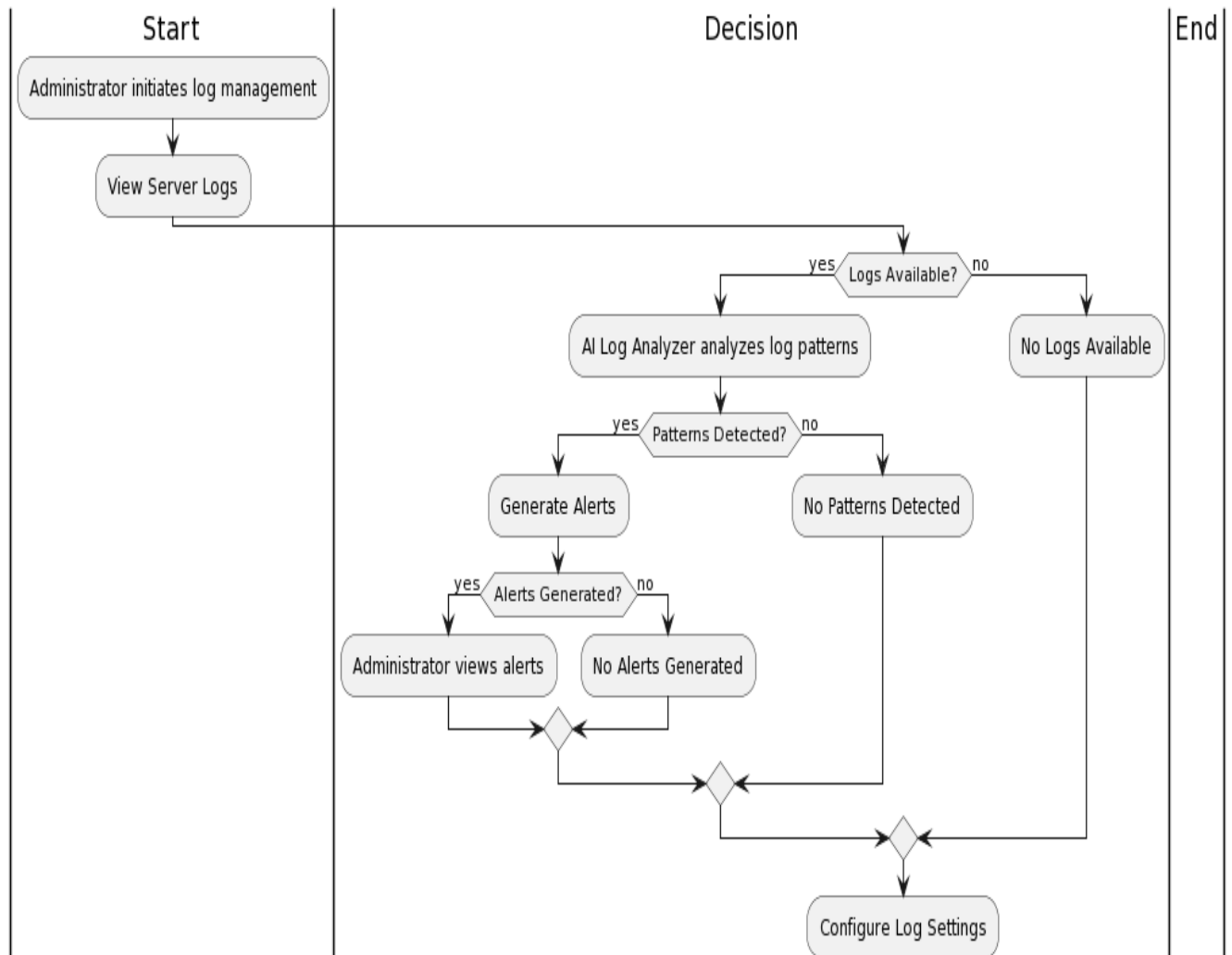


6.2.4 COLLABRATION DIAGRAM

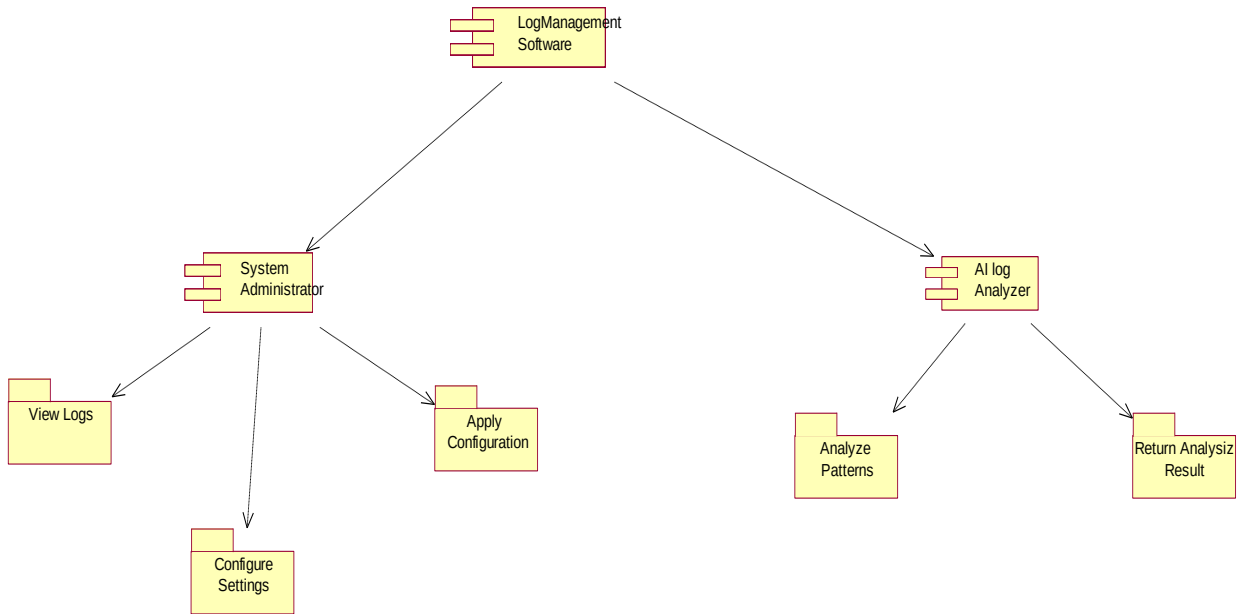


6.2.5 ACTIVITY DIAGRAM

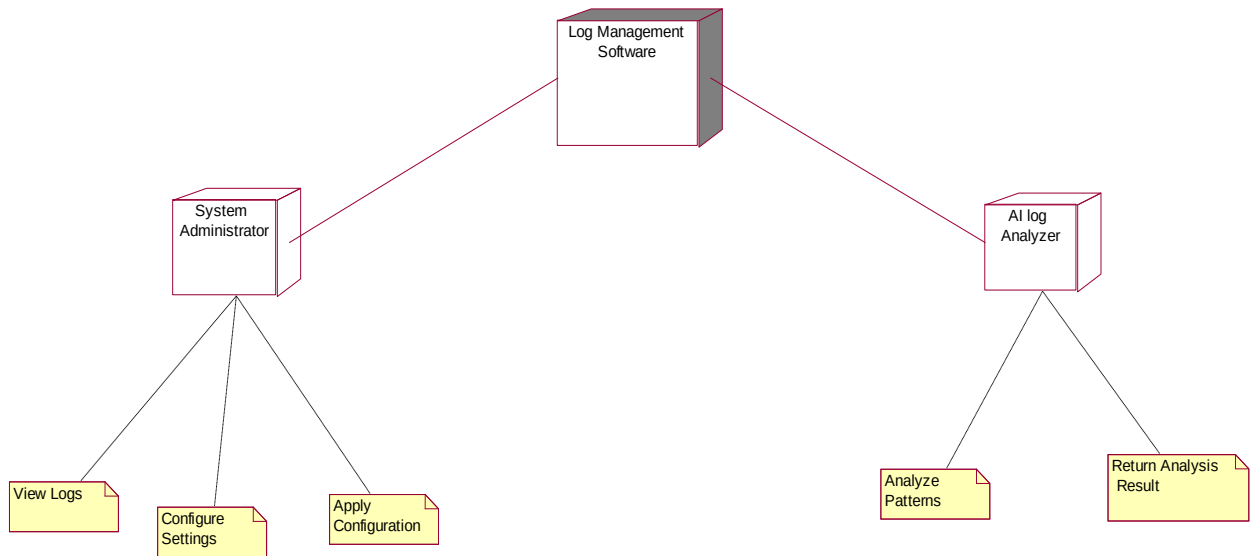
Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.



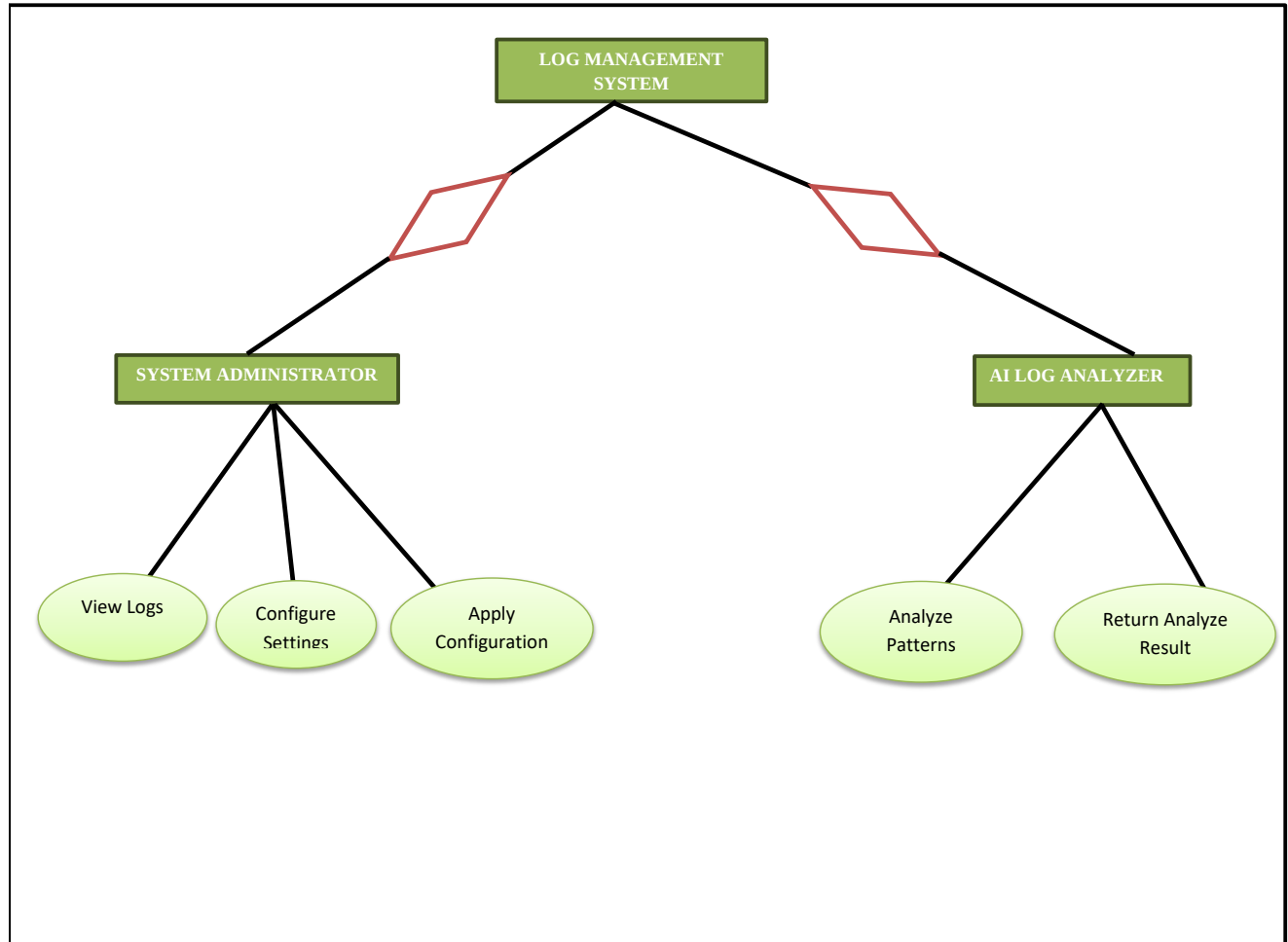
6.2.6 COMPONENT DIAGRAM



6.2.7 DEPLOYMENT DIAGRAM



6.2.8 ER DIAGRAM



6.2.9 DATA SET

Dataset: Error Dataset

```
Errors
how do you express binary literals in python ?
how do i make a menu that does not require the user to press [ enter ] to make a
selection ?
python : what is the difference between ( 1 , 2 , 3 ) and [ 1 , 2 , 3 ], and when should
i use each ?
best way to abstract season / show / episode data
how do i treat an integer as an array of bytes in python ?
why is the pyobjc documentation so bad ?
how to check set of files conform to a naming scheme
how can i graph the lines of code history for git repo ?
how do i create an xml document in python
how can i render a tree structure ( recursive ) using a django template ?
how do you generate dynamic ( parameterized ) unit tests in python ?
why is my instance variable not in __dict__ ?
how to escape os . system () calls ?
what does ** ( double star / asterisk ) and * ( star / asterisk ) do for parameters ?
how can i simply inherit methods from an existing instance ?
how do you check in linux with python if a process is still running ?
```

Ln 1, Col 1 7,012 characters

100%

Unix (LF)

UTF-8

Dataset: Solutions Dataset

Solutions

```
0x12af NUM NUM x100 NUM NUM NUM NUM int ( STR , 2 ) NUM b1100111 NUM b1100111 NUM o27
NUM o27 NUM
raw input () raw input import sys print STR menu NUM ) say foo NUM ) say bar STR answer =
raw input ( STR ) if STR in answer : print STR elif STR in answer : print STR print menu
while lastkey = STR 1 in lastkey : # do stuff
( 1 , 2 , 3 ) [ 1 , 2 , 3 ]
print tvdbinstance [ 1 ][ 23 ][ STR ] # get the name of episode NUM of season NUM tvdb ()
dict () x [ 1 ][ 2 ][ 3 ][ 4 ] = STR if x [ 1 ]. has key ( 2 ): x [ 1 ][ 2 ] = [] self .
data [ show_id ][ season number ][ episode number ][ attribute name ] = STR x [ 3 ][ 24 ]
showcontainer show season episode search () show () __getitem__ __getitem__ has key self .
data raise season not found season () __getitem__ __getitem__ __getitem__ dict dict
self . data [ seas no ][ ep no ][ STR ] = STR __getitem__ __getitem__ __getitem__ dict
( pid , status ) = os . wait () ( exitstatus , signum ) = decode ( status )
nsthread . detachnewthreadselector totarget withobject_ ( STR , self , none )
show name - [ 01x23 ] - episode name . avi show name - [ 01xspecial02 ] - special name .
avi show name - [ 01xextra01 ] - extra name . avi show name - [ 01x23 ] . avi show name /
season NUM / the file . avi show name / season NUM / checker = [ { STR : STR , STR :
STR , STR : check valid () , # runs check valid () on all files STR : 0 # if it returns
```



```

def foo (): STR this function does nothing . STR pass
from tkinter import * import tkmessagebox root = tk () root . withdraw () # todo not if a
window with this title exists tkmessagebox . showinfo ( STR , STR . join ( sys . argv
[ 1 :]))
script_1 . py script_2 . py script_2 . py script_3 . py script_3 . py script_3 . py
script_2 . py os . getcwd ()
select count ( id ), avg ( a ), avg ( b ), avg ( c ), avg ( d ) from tuples ; create
table tuples ( id integer primary key , a integer , b integer , c integer , d integer );
copy tuples from STR delimiter as STR java NUM . 5 threads NUM ms java NUM . 5 NUM ms
python NUM . 5 numpy NUM ms python NUM . 5 NUM ms increased the shared memory available
to postgres to NUM mb increased the working memory to NUM mb disabled connection and
statement logging used a stored procedure via create function language sql
how many days = ( end_date - start_date ). days freqs = defaultdict ( int ) for x in
xrange ( how many responses ): freqs [ int ( how many days * weibullvariate ( 0 . 5 ,
NUM ) ) ] += NUM timeline = [] day = start_date for i , freq in sorted ( freqs . iteritems
()): timeline . append ( ( day , freq ) ) day += timedelta ( days = 1 ) return timeline
d = { STR : STR , STR : STR , STR : STR } this is a this is b this is c
shoes . app do t = para STR button STR do alert STR # when clicked , make an alert t .
replace STR # .. and replace the label STR __main__ : a = myapp () a . ok () # trigger
the clicked action is

```

```

li = [[ 0 , 1 , 2 ],[ 3 , 4 , 5 ],[ 6 , 7 , 8 ]] li2 = [ 0 , 1 , 2 , 3 , 4 , 5 , 6 , 7 ,
8 ] s = STR
text = STR text STR text . decode ( STR ) u STR unicode () unicode ( text ) traceback
( most recent call last ): file STR , line NUM , in < module > unicodedecodeerror : STR
codec can STR utf - 8 STR xe1 STR utf - 8 STR utf - 8 STR xc3 xa1 STR ascii STR t decode
byte NUM xc3 in position NUM : ordinal not in range ( 128 ) unicode ( r , STR ) unicode
( r , STR ) traceback ( most recent call last ): file STR , line NUM , in < module >
TypeError : coercing to unicode : need string or buffer , returnsencoded found
class enum ( object ): STR simple enum class example usage : codes = enum ( STR ) #
codes . baz will be NUM and so on STR def __init__ ( self , names ): for number , name in
enumerate ( names . split ()): setattr ( self , name , number )
class hi : def __init__ ( self ): self . ii = STR self . kk = STR mystery_method ( hi )
[ STR , STR ]
from tkinter import * def quit (): # code to exit root = tk () button ( root , text =
STR , command = quit ). pack () root . mainloop () quit
mytext = self . canvas . create_text ( 5 , NUM , anchor = nw , text = STR )
# i encode the data i STR username STR mandark STR password STR deedee STR http :// www .
forum . com / ucp . php ? mode = login , data ) output = page . read ()
owfs owfs cat cat onewire /{ deviceid }/ log / temperature . 1 onewire /{ deviceid }/ log
/ temperature . 11 history / temperature . 11 onewire /{ deviceid }/ log / temperature . 11

```


CHAPTER-7

INPUT AND OUTPUT DESIGN

7.1 INPUT DESIGN

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy. Input Design considered the following things:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

7.1.1 OBJECTIVES

1.Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.

2. It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities.

3.When the data is entered it will check for its validity. Data can be entered with the help of screens. Appropriate messages are provided as when needed so that the user will not be in maize of instant. Thus the objective of input design is to create an input layout that is easy to follow.

7.2 OUTPUT DESIGN

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

1. Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.

2. Select methods for presenting information.

3. Create document, report, or other formats that contain information produced by the system.

The output form of an information system should accomplish one or more of the following objectives.

- Convey information about past activities, current status or projections of the
- Future.
- Signal important events, opportunities, problems, or warnings.
- Trigger an action.
- Confirm an action.

CHAPTER-8

IMPLEMENTATION

8.1 MODULES

- System Administrator
- Log Management Software
- AI Log Analyzer

8.1.1 MODULE DESCRIPTION

System Administrator

In this module, the System Administrator has initiates log management and can view Server Logs

Log Management Software

In this module, this log management software can view the logs and analysis the patterns and gives the analysis result. It generate Alerts and analysis the result. In this module we can see configure settings.

AI Log Analyzer

In this module, AI log analyzer is analyzing the patterns from log data and gives the analysis result.

CHAPTER-9

SOFTWARE ENVIRONMENT

9.1 PYTHON

Python is a **high-level, interpreted, interactive** and **object-oriented scripting language**. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted:** Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive:** You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented:** Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language:** Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

History of Python

Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.

Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell and other scripting languages.

Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL).

Python is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

Python Features

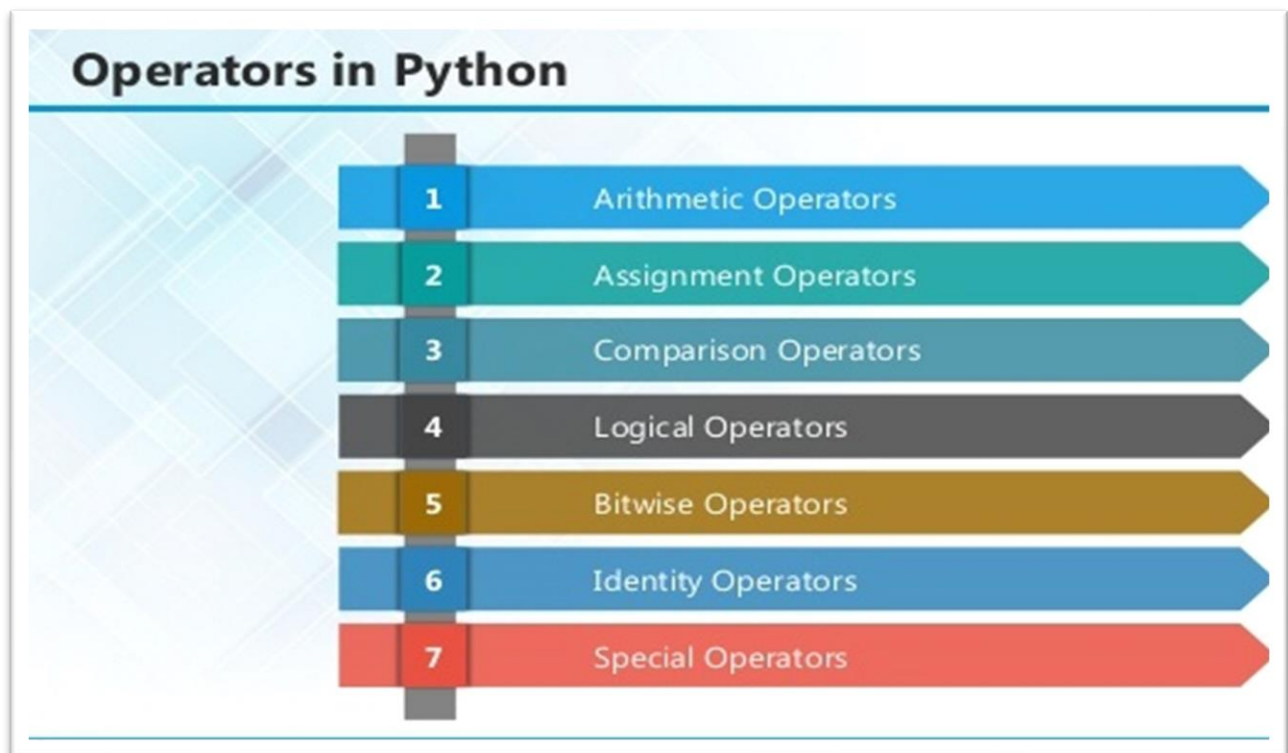
Python's features include:

- **Easy-to-learn:** Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read:** Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain:** Python's source code is fairly easy-to-maintain.
- **A broad standard library:** Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode:** Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable:** Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable:** You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases:** Python provides interfaces to all major commercial databases.
- **GUI Programming:** Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable:** Python provides a better structure and support for large programs than shell scripting.

Python has a big list of good features:

- It supports functional and structured programming methods as well as OOP.

- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.
- IT supports automatic garbage collection.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.



ARITHMETIC OPERATORS

Operator	Description	Example
+ Addition	Adds values on either side of the operator.	$a + b = 30$
- Subtraction	Subtracts right hand operand from left hand operand.	$a - b = -10$
* Multiplication	Multiplies values on either side of the operator	$a * b = 200$
/ Division	Divides left hand operand by right hand operand	$b / a = 2$
% Modulus	Divides left hand operand by right hand operand and returns remainder	$b \% a = 0$
** Exponent	Performs exponential (power) calculation on operators	$a ** b = 10$ to the power 20
//	Floor Division - The division of operands where the result is the quotient in which the digits after the decimal point are removed. But if one of the operands is negative, the result is floored, i.e., rounded away from zero (towards negative infinity):	$9 // 2 = 4$ and $9.0 // 2.0 = 4.0$, - $11 // 3 = -4$, - $11.0 // 3 = -4.0$

ASSIGNMENT OPERATOR

Operator	Description	Example
=	Assigns values from right side operands to left side operand	$c = a + b$ assigns value of $a + b$ into c
+= Add AND	It adds right operand to the left operand and assign the result to left operand	$c += a$ is equivalent to $c = c + a$
-= Subtract AND	It subtracts right operand from the left operand and assign the result to left operand	$c -= a$ is equivalent to $c = c - a$
*= Multiply AND	It multiplies right operand with the left operand and assign the result to left operand	$c *= a$ is equivalent to $c = c * a$
/= Divide AND	It divides left operand with the right operand and assign the result to left operand	$c /= a$ is equivalent to $c = c / a$ $c /= a$ is equivalent to $c = c / a$

%= Modulus AND	It takes modulus using two operands and assign the result to left operand	c %= a is equivalent to c = c % a
**= Exponent AND	Performs exponential (power) calculation on operators and assign value to the left operand	c **= a is equivalent to c = c ** a
//= Floor Division	It performs floor division on operators and assign value to the left operand	c //= a is equivalent to c = c // a

IDENTITY OPERATOR

Operator	Description	Example
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	x is y, here is results in 1 if id(x) equals id(y).
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.	x is not y, here is not results in 1 if id(x) is not equal to id(y)

COMPARISON OPERATOR

Operator	Description	Example
& Binary AND	Operator copies a bit to the result if it exists in both operands	(a & b) (means 0000 1100)
Binary OR	It copies a bit if it exists in either operand.	(a b) = 61 (means 0011 1101)
^ Binary XOR	It copies the bit if it is set in one operand but not both.	(a ^ b) = 49 (means 0011 0001)
~ Binary Ones Complement	It is unary and has the effect of 'flipping' bits.	(~a) = -61 (means 1100 0011 in 2's complement form due to a signed binary number.
<< Binary Left Shift	The left operands value is moved left by the number of bits specified by the right operand.	a << 2 = 240 (means 1111 0000)
>> Binary Right Shift	The left operands value is moved right by the number of bits specified by the right operand.	a >> 2 = 15 (means 0000 1111)

LOGICAL OPERATOR

Operator	Description	Example
and Logical AND	If both the operands are true then condition becomes true.	(a and b) is true.
or Logical OR	If any of the two operands are non-zero then condition becomes true.	(a or b) is true.
not Logical NOT	Used to reverse the logical state of its operand.	Not(a and b) is false.

Membership Operators

Operator	Description	Example
in	Evaluates to true if it finds a variable in the specified sequence and false otherwise.	x in y, here in results in a 1 if x is a member of sequence y.
not in	Evaluates to true if it does not finds a variable in the specified sequence and false otherwise.	x not in y, here not in results in a 1 if x is not a member of sequence y.

Python Operators Precedence

Operator	Description
**	Exponentiation (raise to the power)
~ + -	Complement, unary plus and minus (method names for the last two are +@ and -@)
* / % //	Multiply, divide, modulo and floor division
+ -	Addition and subtraction
>> <<	Right and left bitwise shift
&	Bitwise 'AND'
^	Bitwise exclusive 'OR' and regular 'OR'
<= < > >=	Comparison operators
<> == !=	Equality operators
= %= /= //= -= += *= **=	Assignment operators
is is not	Identity operators
in not in	Membership operators
not or and	Logical operators

LIST

The list is a most versatile data type available in Python which can be written as a list of comma-separated values (items) between square brackets. Important thing about a list is that items in a list need not be of the same type.

Creating a list is as simple as putting different comma-separated values between square brackets. For example –

```
list1 = ['physics', 'chemistry', 1997, 2000];  
list2 = [1, 2, 3, 4, 5];  
list3 = ["a", "b", "c", "d"]
```

Basic List Operations

Lists respond to the + and * operators much like strings; they mean concatenation and repetition here too, except that the result is a new list, not a string.

Python Expression	Results	Description
len([1, 2, 3])	3	Length
[1, 2, 3] + [4, 5, 6]	[1, 2, 3, 4, 5, 6]	Concatenation
['Hi!'] * 4	['Hi!', 'Hi!', 'Hi!', 'Hi!']	Repetition
3 in [1, 2, 3]	True	Membership
for x in [1, 2, 3]: print x,	1 2 3	Iteration

Built-in List Functions & Methods:

Python includes the following list functions –

SN	Function with Description
1	<u><code>cmp(list1, list2)</code></u> Compares elements of both lists.
2	<u><code>len(list)</code></u> Gives the total length of the list.
3	<u><code>max(list)</code></u> Returns item from the list with max value.
4	<u><code>min(list)</code></u> Returns item from the list with min value.
5	<u><code>list(seq)</code></u> Converts a tuple into list.

Python includes following list methods

SN	Methods with Description
1	<u><code>list.append(obj)</code></u> Appends object obj to list
2	<u><code>list.count(obj)</code></u> Returns count of how many times obj occurs in list
3	<u><code>list.extend(seq)</code></u> Appends the contents of seq to list
4	<u><code>list.index(obj)</code></u> Returns the lowest index in list that obj appears
5	<u><code>list.insert(index, obj)</code></u> Inserts object obj into list at offset index
6	<u><code>list.pop(obj=list[-1])</code></u> Removes and returns last object or obj from list
7	<u><code>list.remove(obj)</code></u> Removes object obj from list
8	<u><code>list.reverse()</code></u> Reverses objects of list in place
9	<u><code>list.sort([func])</code></u> Sorts objects of list, use compare function if given

3.2 TUPLES

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

Creating a tuple is as simple as putting different comma-separated values. Optionally we can put these comma-separated values between parentheses also. For example –

```
tup1 = ('physics', 'chemistry', 1997, 2000);  
tup2 = (1, 2, 3, 4, 5);  
tup3 = "a", "b", "c", "d";
```

The empty tuple is written as two parentheses containing nothing –

```
tup1 = ();
```

To write a tuple containing a single value you have to include a comma, even though there is only one value –

```
tup1 = (50,);
```

Like string indices, tuple indices start at 0, and they can be sliced, concatenated, and so on.

- Accessing Values in Tuples:

To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index. For example –

```
tup1 = ('physics', 'chemistry', 1997, 2000);  
tup2 = (1, 2, 3, 4, 5, 6, 7);  
print "tup1[0]: ", tup1[0]  
print "tup2[1:5]: ", tup2[1:5]
```

When the code is executed, it produces the following result –

```
tup1[0]: physics  
tup2[1:5]: [2, 3, 4, 5]
```


Updating Tuples:

Tuples are immutable which means you cannot update or change the values of tuple elements. We are able to take portions of existing tuples to create new tuples as the following example demonstrates –

```
tup1 = (12, 34.56);  
tup2 = (' abc ', 'xyz ');  
tup3 = tup1 + tup2;  
print tup3
```

When the above code is executed, it produces the following result –

```
(12, 34.56, 'abc', 'xyz')
```

Delete Tuple Elements

Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded.

To explicitly remove an entire tuple, just use the **del** statement. For example:

```
tup = ('physics', 'chemistry', 1997, 2000);  
print tup  
del tup;  
print "After deleting tup : "  
print tup
```

Basic Tuples Operations:

Python Expression	Results	Description

<code>len((1, 2, 3))</code>	3	Length
<code>(1, 2, 3) + (4, 5, 6)</code>	<code>(1, 2, 3, 4, 5, 6)</code>	Concatenation
<code>('Hi!') * 4</code>	<code>('Hi!', 'Hi!', 'Hi!', 'Hi!')</code>	Repetition
<code>3 in (1, 2, 3)</code>	True	Membership
<code>for x in (1, 2, 3): print x,</code>	1 2 3	Iteration

Built-in Tuple Functions

SN	Function with Description
1	cmp(tuple1, tuple2): Compares elements of both tuples.
2	len(tuple): Gives the total length of the tuple.
3	max(tuple): Returns item from the tuple with max value.
4	min(tuple): Returns item from the tuple with min value.
5	tuple(seq): Converts a list into tuple.

DICTIONARY

Each key is separated from its value by a colon (:), the items are separated by commas, and the whole thing is enclosed in curly braces. An empty dictionary without any items is written with just two curly braces, like this: {}.

Keys are unique within a dictionary while values may not be. The values of a dictionary can be of any type, but the keys must be of an immutable data type such as strings, numbers, or tuples.

Accessing Values in Dictionary:

To access dictionary elements, you can use the familiar square brackets along with the key to obtain its value. Following is a simple example –

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}

print "dict['Name']: ", dict['Name']
print "dict['Age']: ", dict['Age']
```

Result –

```
dict['Name']: Zara
dict['Age']: 7
```

Updating Dictionary

We can update a dictionary by adding a new entry or a key-value pair, modifying an existing entry, or deleting an existing entry as shown below in the simple example –

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}

dict['Age'] = 8; # update existing entry
dict['School'] = "DPS School"; # Add new entry
print "dict['Age']: ", dict['Age']
print "dict['School']: ", dict['School']
```

Result –

```
dict['Age']: 8
dict['School']: DPS School
```

Delete Dictionary Elements

We can either remove individual dictionary elements or clear the entire contents of a dictionary. You can also delete entire dictionary in a single operation.

To explicitly remove an entire dictionary, just use the **del** statement. Following is a simple example

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}

del dict['Name']; # remove entry with key 'Name'

dict.clear();    # remove all entries in dict

del dict;        # delete entire dictionary

print "dict['Age']: ", dict['Age']
print "dict['School']: ", dict['School']
```

Built-in Dictionary Functions & Methods—

Python includes the following dictionary functions —

SN	Function with Description
1	<u>cmp(dict1, dict2)</u> Compares elements of both dict.
2	<u>len(dict)</u> Gives the total length of the dictionary. This would be equal to the number of items in the dictionary.

3	<u>str(dict)</u> Produces a printable string representation of a dictionary
4	<u>type(variable)</u> Returns the type of the passed variable. If passed variable is dictionary, then it would return a dictionary type.

Python includes following dictionary methods –

SN	Methods with Description
1	dict.clear(): Removes all elements of dictionary <i>dict</i>
2	dict. Copy(): Returns a shallow copy of dictionary <i>dict</i>
3	dict.fromkeys(): Create a new dictionary with keys from seq and values set to <i>value</i> .
4	dict.get(key, default=None): For <i>key</i> key, returns value or default if key not in dictionary
5	dict.has_key(key): Returns <i>true</i> if key in dictionary <i>dict</i> , <i>false</i> otherwise
6	dict.items(): Returns a list of <i>dict</i> 's (key, value) tuple pairs
7	dict.keys(): Returns list of dictionary dict's keys

8	dict.setdefault(key, default=None): Similar to get(), but will set dict[key]=default if <i>key</i> is not already in dict
9	dict.update(dict2): Adds dictionary <i>dict2</i> 's key-values pairs to <i>dict</i>
10	dict.values(): Returns list of dictionary <i>dict</i> 's values

A function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing. Python gives you many built-in functions like print(), etc. but you can also create your own functions. These functions are called *user-defined functions*.

Defining a Function

Simple rules to define a function in Python.

- Function blocks begin with the keyword `def` followed by the function name and parentheses `(( ))`.
- Any input parameters or arguments should be placed within these parentheses. You can also define parameters inside these parentheses.
- The first statement of a function can be an optional statement - the documentation string of the function or *docstring*.
- The code block within every function starts with a colon `(:)` and is indented.
- The statement `return [expression]` exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as `return None`.

```
def functionname( parameters ):
    "function_docstring"
```

```
function_suite  
  
return [expression]
```

Calling a Function

Defining a function only gives it a name, specifies the parameters that are to be included in the function and structures the blocks of code. Once the basic structure of a function is finalized, you can execute it by calling it from another function or directly from the Python prompt. Following is the example to call printme() function –

```
# Function definition is here  
  
def printme( str ):      
    "This prints a passed string into this function"  
    print str  
    return;  
  
# Now you can call printme function  
  
printme("I'm first call to user defined function!")  
printme("Again second call to the same function")
```

When the above code is executed, it produces the following result –

```
I'm first call to user defined function!  
Again second call to the same function
```

Function Arguments

You can call a function by using the following types of formal arguments:

- Required arguments
- Keyword arguments
- Default arguments
- Variable-length arguments

Scope of Variables

All variables in a program may not be accessible at all locations in that program. This depends on where you have declared a variable.

The scope of a variable determines the portion of the program where you can access a particular identifier. There are two basic scopes of variables in Python –

Global variables

Local variables

Global vs. Local variables

Variables that are defined inside a function body have a local scope, and those defined outside have a global scope.

This means that local variables can be accessed only inside the function in which they are declared, whereas global variables can be accessed throughout the program body by all functions. When you call a function, the variables declared inside it are brought into scope. Following is a simple example –

—


```

total = 0; # This is global variable.

# Function definition is here
def sum( arg1, arg2 ):
    # Add both the parameters and return them."
    total = arg1 + arg2; # Here total is local variable.
    print "Inside the function local total : ",total
    return total;
sum( 10, 20 );
print "Outside the function global total : ",total

```

Result –

```

Inside the function local total : 30
Outside the function global total : 0

```

A module allows you to logically organize your Python code. Grouping related code into a module makes the code easier to understand and use. A module is a Python object with arbitrarily named attributes that you can bind and reference. Simply, a module is a file consisting of Python code. A module can define functions, classes and variables. A module can also include runnable code.

Example:

The Python code for a module named *aname* normally resides in a file named *aname.py*. Here's an example of a simple module, *support.py*

```

def print_func( par ):
    print "Hello : ", par
    return

```

The *import* Statement

The *import* has the following syntax:

```
import module1[, module2[,... moduleN]
```

When the interpreter encounters an import statement, it imports the module if the module is present in the search path. A search path is a list of directories that the interpreter searches before importing a module. For example, to import the module `support.py`, you need to put the following command at the top of the script –

A module is loaded only once, regardless of the number of times it is imported. This prevents the module execution from happening over and over again if multiple imports occur.

Packages in Python

A package is a hierarchical file directory structure that defines a single Python application environment that consists of modules and sub packages and sub-sub packages.

Consider a file *Pots.py* available in *Phone* directory. This file has following line of source code –

```
def Pots():  
    print "I'm Pots Phone"
```

Similar way, we have another two files having different functions with the same name as above –

- *Phone/Isdn.py* file having function `Isdn()`
- *Phone/G3.py* file having function `G3()`

Now, create one more file `__init__.py` in *Phone* directory –

- *Phone/__init__.py*

To make all of your functions available when you've imported Phone, to put explicit import statements in `__init__.py` as follows –

```
from Pots import Pots
from Isdn import Isdn
from G3 import G3
```

After you add these lines to `__init__.py`, you have all of these classes available when you import the Phone package.

```
# Now import your Phone Package.
import Phone
Phone.Pots()
Phone.Isdn()
Phone.G3()
```

RESULT:

```
I'm Pots Phone
I'm 3G Phone
I'm ISDN Phone
```

In the above example, we have taken example of a single functions in each file, but you can keep multiple functions in your files. You can also define different Python classes in those files and then you can create your packages out of those classes.

This chapter covers all the basic I/O functions available in Python.

Printing to the Screen

The simplest way to produce output is using the *print* statement where you can pass zero or more expressions separated by commas. This function converts the expressions you pass into a string and

writes the result to standard output as follows –

```
print "Python is really a great language,","isn't it?"
```

Result:

```
Python is really a great language, isn't it?
```

Reading Keyboard Input

Python provides two built-in functions to read a line of text from standard input, which by default comes from the keyboard. These functions are –

- `raw_input`
- `input`

The `raw_input` Function

The `raw_input([prompt])` function reads one line from standard input and returns it as a string (removing the trailing newline).

```
str = raw_input("Enter your input: ");  
print "Received input is : ", str
```

This prompts you to enter any string and it would display same string on the screen. When I typed "Hello Python!", its output is like this –

```
Enter your input: Hello Python  
Received input is : Hello Python
```

The `input` Function

The `input([prompt])` function is equivalent to `raw_input`, except that it assumes the input is a valid Python expression and returns the evaluated result to you.

```
str = input("Enter your input: ");  
print "Received input is : ", str
```

This would produce the following result against the entered input –

```
Enter your input: [x*5 for x in range(2,10,2)]
```

```
Recieved input is : [10, 20, 30, 40]
```

Opening and Closing Files

Until now, you have been reading and writing to the standard input and output. Now, we will see how to use actual data files.

Python provides basic functions and methods necessary to manipulate files by default. You can do most of the file manipulation using a **file** object.

The *open* Function

Before you can read or write a file, you have to open it using Python's built-in *open()* function. This function creates a **file** object, which would be utilized to call other support methods associated with it.

Syntax

```
file object = open(file_name [, access_mode][, buffering])
```

Here are parameter details:

- **file_name:** The file_name argument is a string value that contains the name of the file that you want to access.
- **access_mode:** The access_mode determines the mode in which the file has to be opened, i.e., read, write, append, etc. A complete list of possible values is given below in the table. This is optional parameter and the default file access mode is read (r).
- **buffering:** If the buffering value is set to 0, no buffering takes place. If the buffering value is 1, line buffering is performed while accessing a file. If you specify the buffering value as an integer greater than 1, then buffering action is performed with the indicated buffer size. If negative, the buffer size is the system default(default behavior).

Here is a list of the different modes of opening a file –

Modes	Description
r	Opens a file for reading only. The file pointer is placed at the beginning of the file. This is the default mode.
rb	Opens a file for reading only in binary format. The file pointer is placed at the beginning of the file. This is the default mode.
r+	Opens a file for both reading and writing. The file pointer placed at the beginning of the file.
rb+	Opens a file for both reading and writing in binary format. The file pointer placed at the beginning of the file.
w	Opens a file for writing only. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
wb	Opens a file for writing only in binary format. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
w+	Opens a file for both writing and reading. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
wb+	Opens a file for both writing and reading in binary format. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
a	Opens a file for appending. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.

ab	Opens a file for appending in binary format. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
a+	Opens a file for both appending and reading. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.
ab+	Opens a file for both appending and reading in binary format. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.

The *file* Object Attributes

Once a file is opened and you have one *file* object, you can get various information related to that file.

Here is a list of all attributes related to file object:

Attribute	Description
file.closed	Returns true if file is closed, false otherwise.
file.mode	Returns access mode with which file was opened.
file.name	Returns name of the file.
file.softspace	Returns false if space explicitly required with print, true otherwise.

Example

```
# Open a file
fo = open("foo.txt", "wb")
print "Name of the file: ", fo.name
```

```
print "Closed or not : ", fo.closed
print "Opening mode : ", fo.mode
print "Softspace flag : ", fo.softspace
```

This produces the following result –

```
Name of the file: foo.txt
Closed or not : False
Opening mode : wb
Softspace flag : 0
```

The `close()` Method

The `close()` method of a *file* object flushes any unwritten information and closes the file object, after which no more writing can be done. Python automatically closes a file when the reference object of a file is reassigned to another file. It is a good practice to use the `close()` method to close a file

Syntax

```
fileObject.close();
```

Example

```
# Open a file
fo = open("foo.txt", "wb")
print "Name of the file: ", fo.name
# Close opened file
fo.close()
```

Result –

```
Name of the file: foo.txt
```

Reading and Writing Files

The *file* object provides a set of access methods to make our lives easier. We would see how to use `read()` and `write()` methods to read and write files.

The `write()` Method

The `write()` method writes any string to an open file. It is important to note that Python strings can have binary data and not just text. The `write()` method does not add a newline character ('\n') to the end of the string **Syntax**

```
fileObject.write(string);
```

Here, passed parameter is the content to be written into the opened file. **Example**

```
# Open a file
fo = open("foo.txt", "wb")
fo.write("Python is a great language.\nYeah its great!!\n");

# Close opened file
fo.close()
```

The above method would create `foo.txt` file and would write given content in that file and finally it would close that file. If you would open this file, it would have following content.

```
Python is a great language.
Yeah its great!!
```

The `read()` Method

The `read()` method reads a string from an open file. It is important to note that Python strings can have binary data. apart from text data.

Syntax

```
fileObject.read([count]);
```

Here, passed parameter is the number of bytes to be read from the opened file. This method starts reading from the beginning of the file and if `count` is missing, then it tries to read as much as possible, maybe until the end of file.

Example

Let's take a file *foo.txt*, which we created above.

```
# Open a file
fo = open("foo.txt", "r+")
str = fo.read(10);
print "Read String is : ", str
# Close opened file
fo.close()
```

This produces the following result –

```
Read String is : Python is
```

File Positions

The *tell()* method tells you the current position within the file; in other words, the next read or write will occur at that many bytes from the beginning of the file.

32

The *seek(offset[, from])* method changes the current file position. The *offset* argument indicates the number of bytes to be moved. The *from* argument specifies the reference position from where the bytes are to be moved.

If *from* is set to 0, it means use the beginning of the file as the reference position and 1 means use the current position as the reference position and if it is set to 2 then the end of the file would be taken as the reference position.

Example

Let us take a file *foo.txt*, which we created above.

```
# Open a file
fo = open("foo.txt", "r+")
str = fo.read(10);
```

```

print "Read String is : ", str

# Check current position
position = fo.tell();
print "Current file position : ", position

# Reposition pointer at the beginning once again
position = fo.seek(0, 0);
str = fo.read(10);
print "Again read String is : ", str

# Close opened file
fo.close()

```

This produces the following result –

```

Read String is : Python is
Current file position : 10
Again read String is : Python is

```

Renaming and Deleting Files

Python **os** module provides methods that help you perform file-processing operations, such as renaming and deleting files.

To use this module you need to import it first and then you can call any related functions.

The `rename()` Method

The `rename()` method takes two arguments, the current filename and the new filename.

Syntax

```
os.rename(current_file_name, new_file_name)
```

Example

Following is the example to rename an existing file *test1.txt*:

```
import os

# Rename a file from test1.txt to test2.txt
os.rename( "test1.txt", "test2.txt" )
```

The *remove()* Method

You can use the *remove()* method to delete files by supplying the name of the file to be deleted as the argument.

Syntax

```
os.remove(file_name)
```

Example

Following is the example to delete an existing file *test2.txt* –

```
#!/usr/bin/python
import os

# Delete file test2.txt
os.remove("test2.txt")
```

Directories in Python

All files are contained within various directories, and Python has no problem handling these too. The **os** module has several methods that help you create, remove, and change directories.

The *mkdir()* Method

You can use the *mkdir()* method of the **os** module to create directories in the current directory. You need to supply an argument to this method which contains the name of the directory to be created.

Syntax

```
os.mkdir("newdir")
```

Example

Following is the example to create a directory *test* in the current directory –

```
#!/usr/bin/python
import os

# Create a directory "test"
os.mkdir("test")
```

The *chdir()* Method

You can use the *chdir()* method to change the current directory. The *chdir()* method takes an argument, which is the name of the directory that you want to make the current directory.

Syntax

```
os.chdir("newdir")
```

Example

Following is the example to go into *"/home/newdir"* directory –

```
#!/usr/bin/python
import os

# Changing a directory to "/home/newdir"
os.chdir("/home/newdir")
```

The *getcwd()* Method

The *getcwd()* method displays the current working directory.

Syntax

```
os.getcwd()
```

Example

Following is the example to give current directory –

```
import os

# This would give location of the current directory

os.getcwd()
```

The *rmdir()* Method

The *rmdir()* method deletes the directory, which is passed as an argument in the method.

Before removing a directory, all the contents in it should be removed.

Syntax:

```
os.rmdir('dirname')
```

Example

Following is the example to remove `"/tmp/test"` directory. It is required to give fully qualified name of the directory, otherwise it would search for that directory in the current directory.

```
import os

# This would remove "/tmp/test" directory.

os.rmdir( "/tmp/test" )
```

File & Directory Related Methods

There are three important sources, which provide a wide range of utility methods to handle and manipulate files & directories on Windows and Unix operating systems. They are as follows –

- File Object Methods: The *file* object provides functions to manipulate files.
- OS Object Methods: This provides methods to process files as well as directories.

Python provides two very important features to handle any unexpected error in your Python programs and to add debugging capabilities in them –

- **Exception Handling:** This would be covered in this tutorial. Here is a list standard Exceptions available in Python: [Standard Exceptions](#).
- **Assertions:** This would be covered in [Assertions in Python](#)

List of Standard Exceptions –

EXCEPTION NAME	DESCRIPTION
Exception	Base class for all exceptions
StopIteration	Raised when the next() method of an iterator does not point to any object.
SystemExit	Raised by the sys.exit() function.
StandardError	Base class for all built-in exceptions except StopIteration and SystemExit.
ArithmeticError	Base class for all errors that occur for numeric calculation.
OverflowError	Raised when a calculation exceeds maximum limit for a numeric type.
FloatingPointError	Raised when a floating point calculation fails.
ZeroDivisionError	Raised when division or modulo by zero takes place for all numeric types.

AssertionError	Raised in case of failure of the Assert statement.
AttributeError	Raised in case of failure of attribute reference or assignment.
EOFError	Raised when there is no input from either the raw_input() or input() function and the end of file is reached.
ImportError	Raised when an import statement fails.
KeyboardInterrupt	Raised when the user interrupts program execution, usually by pressing Ctrl+c.
LookupError	Base class for all lookup errors.
IndexError	Raised when an index is not found in a sequence.
KeyError	Raised when the specified key is not found in the dictionary.
NameError	Raised when an identifier is not found in the local or global namespace.
UnboundLocalError	Raised when trying to access a local variable in a function or method but no value has been assigned to it.
EnvironmentError	Base class for all exceptions that occur outside the Python environment.

IOError IOError	Raised when an input/ output operation fails, such as the print statement or the open() function when trying to open a file that does not exist. Raised for operating system-related errors.
SyntaxError IndentationError	Raised when there is an error in Python syntax. Raised when indentation is not specified properly.
SystemError	Raised when the interpreter finds an internal problem, but when this error is encountered the Python interpreter does not exit.
SystemExit	Raised when Python interpreter is quit by using the sys.exit() function. If not handled in the code, causes the interpreter to exit.
TypeError	Raised when an operation or function is attempted that is invalid for the specified data type.
ValueError	Raised when the built-in function for a data type has the valid type of arguments, but the arguments have invalid values specified.
RuntimeError	Raised when a generated error does not fall into any category.
NotImplementedError	Raised when an abstract method that needs to be implemented in an inherited class is not actually implemented.

What is Exception?

An exception is an event, which occurs during the execution of a program that disrupts the normal flow of the program's instructions. In general, when a Python script encounters a situation that it cannot cope with, it raises an exception. An exception is a Python object that represents an error.

When a Python script raises an exception, it must either handle the exception immediately otherwise it terminates and quits.

Handling an exception

If you have some *suspicious* code that may raise an exception, you can defend your program by placing the suspicious code in a **try:** block. After the try: block, include an **except:** statement, followed by a block of code which handles the problem as elegantly as possible.

The Python standard for database interfaces is the Python DB-API. Most Python database interfaces adhere to this standard.

You can choose the right database for your application. Python Database API supports a wide range of database servers such as –

- GadFly
- mSQL
- MySQL
- PostgreSQL
- Microsoft SQL Server 2000
- Informix
- Interbase
- Oracle
- Sybase

The DB API provides a minimal standard for working with databases using Python structures and syntax wherever possible. This API includes the following:

- Importing the API module.
- Acquiring a connection with the database.
- Issuing SQL statements and stored procedures.
- Closing the connection

9.2 SOURCE CODE

MANAGE.PY

```
#!/usr/bin/env python
```

```
import os
```

```
import sys
```

```
if __name__ == '__main__':
```

```
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'AILog.settings')
```

```
    try:
```

```
        from django.core.management import execute_from_command_line
```

```
    except ImportError as exc:
```

```
        raise ImportError(
```

```
            "Couldn't import Django. Are you sure it's installed and "
```

```
            "available on your PYTHONPATH environment variable? Did you "
```

```
            "forget to activate a virtual environment?"
```

```
        ) from exc
```

```
execute_from_command_line(sys.argv)
```

VIEWS.PY

```
from django.shortcuts import render
```

```
from django.template import RequestContext
```

```
from django.contrib import messages
```

```
from django.http import HttpResponse
```

```
import os
```

```
import pandas as pd
```

```
import numpy as np
```

```
import pickle
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
import pymysql
```

```
import os
```

```
from numpy import dot
```

```
from numpy.linalg import norm
```

```
from django.core.files.storage import FileSystemStorage
```

```
from string import punctuation
```

```
from nltk.corpus import stopwords
```

```
import nltk
```

```
from nltk.stem import WordNetLemmatizer
```

```
from nltk.stem import PorterStemmer
```

```
import requests
```

```
from bs4 import BeautifulSoup
```

```
import re
```

```
global uname, solution, vectorizer, tfidf
```

```
stop_words = set(stopwords.words('english'))
```

```
lemmatizer = WordNetLemmatizer()
```

```
ps = PorterStemmer()
```

```
def cleanText(doc):
```

```
    tokens = doc.split()
```

```
    table = str.maketrans("", "", punctuation)
```

```
    tokens = [w.translate(table) for w in tokens]
```

```
    tokens = [word for word in tokens if word.isalpha()]
```

```
    tokens = [w for w in tokens if not w in stop_words]
```

```
    tokens = [word for word in tokens if len(word) > 1]
```

```
    tokens = [ps.stem(token) for token in tokens]
```

```
    tokens = [lemmatizer.lemmatize(token) for token in tokens]
```

```
    tokens = ' '.join(tokens)
```

```
    return tokens
```

```
def SearchSolution(request):
```

```
    if request.method == 'GET':
```

```
        return render(request, 'SearchSolution.html', {})
```

```
def getLink(error):
```

```
    search_link = "No Google Link Found"
```

```

page = requests.get("https://www.google.com/search?q="+error)
soup = BeautifulSoup(page.content)
links = soup.findAll("a")
for link in soup.find_all("a", href=re.compile("(?<=/url\?q=)(htt://.)")):
    search_link = re.split(":(?=http)", link["href"].replace("/url?q=", ""))
    if len(search_link) > 0:
        search_link = search_link[0]
        break
return search_link

```

```

def SearchSolutionAction(request):
    if request.method == 'POST':
        global solution, vectorizer, tfidf
        error = request.POST.get('t1', False)
        link = getLink(error)
        test_query = error.strip().lower()
        test_query = cleanText(test_query)
        testData = vectorizer.transform([test_query]).toarray()
        testData = testData[0]
        accuracy = 0
        solution_index = -1
        for i in range(len(tfidf)):
            predict_score = dot(tfidf[i], testData)/(norm(tfidf[i])*norm(testData))
            if predict_score > accuracy:
                accuracy = predict_score

```

```

        solution_index = i

output = "Error = "+error+"<br/>Solution = Unable to predict solution. Please try other query"

if solution_index != -1:

    output = "Error = "+error+"<br/>Solution = "+solution[solution_index]

    output += '<br/>Google Link = <a href="'+link+'" target="_blank">'+link+'</a>'

context= {'data':output}

return render(request, 'SearchSolution.html', context)

```

```
def TrainModel(request):
```

```
    if request.method == 'GET':
```

```
        global solution, vectorizer, tfidf
```

```
        accuracy = 0
```

```
        error = pd.read_csv("Dataset/ErrorQuestion.txt", usecols=['Errors'])
```

```
        error = error.values.ravel()
```

```
        solution = pd.read_csv("Dataset/solutions.txt", encoding='iso-8859-1', usecols=['Solutions'])
```

```
        solution = solution.values.ravel()
```

```
        clean = []
```

```
        for i in range(len(error)):
```

```
            data = error[i].strip().lower()
```

```
            data = cleanText(data)
```

```
            clean.append(data)
```

```
        vectorizer = TfidfVectorizer(stop_words=stop_words, use_idf=True, smooth_idf=False,
norm=None, decode_error='replace')
```

```
        tfidf = vectorizer.fit_transform(clean).toarray()
```

```

test_query = "sql aggregate functions so much slower python and java ( or poor man s olap )"
test_query = test_query.strip().lower()
test_query = cleanText(test_query)
testData = vectorizer.transform([test_query]).toarray()
testData = testData[0]
for i in range(len(tfidf)):
    predict_score = dot(tfidf[i], testData)/(norm(tfidf[i])*norm(testData))
    if predict_score > accuracy:
        accuracy = predict_score
context= {'data':'AI Training Completed with Accuracy : '+str(accuracy)}
return render(request, 'AdminScreen.html', context)

```

```

def Signup(request):
    if request.method == 'GET':
        return render(request, 'Signup.html', {})

```

```

def index(request):
    if request.method == 'GET':
        return render(request, 'index.html', {})

```

```

def UserLogin(request):
    if request.method == 'GET':
        return render(request, 'UserLogin.html', {})

```

```

def AdminLogin(request):

```



```
if request.method == 'GET':  
    return render(request, 'AdminLogin.html', {})
```

```
def AdminLoginAction(request):
```

```
    if request.method == 'POST':  
        global userid  
        user = request.POST.get('t1', False)  
        password = request.POST.get('t2', False)  
        if user == "admin" and password == "admin":  
            context= {'data':'Welcome '+user}  
            return render(request, 'AdminScreen.html', context)  
        else:  
            context= {'data':'Invalid Login'}  
            return render(request, 'AdminLogin.html', context)
```

```
def UserLoginAction(request):
```

```
    if request.method == 'POST':  
        global uname  
        username = request.POST.get('t1', False)  
        password = request.POST.get('t2', False)  
        index = 0  
        con = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = "", database =  
'ailog',charset='utf8')  
        with con:  
            cur = con.cursor()
```

```

cur.execute("select * FROM register")

rows = cur.fetchall()

for row in rows:

    if row[0] == username and password == row[1]:

        uname = username

        index = 1

        break

if index == 1:

    context= {'data':'welcome '+username}

    return render(request, 'UserScreen.html', context)

else:

    context= {'data':'login failed'}

    return render(request, 'UserLogin.html', context)


def SignupAction(request):

    if request.method == 'POST':

        username = request.POST.get('t1', False)

        password = request.POST.get('t2', False)

        contact = request.POST.get('t3', False)

        email = request.POST.get('t4', False)

        address = request.POST.get('t5', False)

        status = "none"

        con = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = "", database =
'ailog',charset='utf8')

```

```

with con:

    cur = con.cursor()

    cur.execute("select * FROM register")

    rows = cur.fetchall()

    for row in rows:

        if row[0] == username:

            status = "Username already exists"

            break

    if status == "none":

        db_connection = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = "",
database = 'ailog',charset='utf8')

        db_cursor = db_connection.cursor()

        student_sql_query = "INSERT INTO register(username,password,contact,email,address)
VALUES('"+username+"','"+password+"','"+contact+"','"+email+"','"+address+"')"

        db_cursor.execute(student_sql_query)

        db_connection.commit()

        print(db_cursor.rowcount, "Record Inserted")

        if db_cursor.rowcount == 1:

            status = "Signup Process Completed. You can Login now"

            context= {'data': status}

            return render(request, 'Signup.html', context)

def ViewUser(request):

    if request.method == 'GET':

        output = "

```

```

        output+=<table border=1 align=center width=100%><tr><th><font size=""
color="black">Username</th><th><font size="" color="black">Password</th><th><font size=""
color="black">Contact No</th>

```

```

        output+=<th><font size="" color="black">Email ID</th><th><font size=""
color="black">Address</th></tr>

```

```

con = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = "", database =
'ailog',charset='utf8')

```

with con:

```

cur = con.cursor()

```

```

cur.execute("select * from register")

```

```

rows = cur.fetchall()

```

```

output+=<tr>

```

for row in rows:

```

        output+=<td><font size="" color="black">'+row[0]+'</td><td><font size=""
color="black">'+str(row[1])+'</td><td><font size="" color="black">'+row[2]+'</td><td><font
size="" color="black">'+row[3]+'</td><td><font size="" color="black">'+row[4]+'</td></tr>

```

```

output+= "</table></br></br></br></br>"

```

```

context= {'data':output}

```

```

return render(request, 'AdminScreen.html', context)

```

CHAPTER-10

RESULTS/DISCUSSION

10.1 SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

TYPES OF TESTS

UNIT TESTING

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

INTEGRATION TESTING

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

FUNCTIONAL TEST

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

- Valid Input : identified classes of valid input must be accepted.
- Invalid Input : identified classes of invalid input must be rejected.
- Functions : identified functions must be exercised.
- Output : identified classes of application outputs must be exercised.
- Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

SYSTEM TEST

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration-oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

WHITE BOX TESTING

White Box Testing is a testing in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is used to test areas that cannot be reached from a black box level.

BLACK BOX TESTING

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

UNIT TESTING

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

TEST STRATEGY AND APPROACH

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

ACCEPTANCE TESTING

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

10.1.1 Test Cases

Test case1:

Test case for Login form:

FUNCTION:	LOGIN
EXPECTED RESULTS:	Should Validate the user and check his existence in database
ACTUAL RESULTS:	Validate the user and checking the user against the database
LOW PRIORITY	No
HIGH PRIORITY	Yes

Test case2:

Test case for User Registration form:

FUNCTION	USER REGISTRATION
EXPECTED RESULTS:	Should check if all the fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are filled by user or not through validations and saving user.

LOW PRIORITY	No
HIGH PRIORITY	Yes

Test case3:

Test case for Change Password:

When the old password does not match with the new password , then this results in displaying an error message as “ OLD PASSWORD DOES NOT MATCH WITH THE NEW PASSWORD”.

Test case 4:

Test case for Forget Password:

When a user forgets his password he is asked to enter Login name, ZIP code, Mobile number. If these are matched with the already stored ones then user will get his Original password.

Module	Functionality	Test Case	Expected Results	Actual Results	Result	Priority
User	Login Usecase	1. Navigate To Www.Sample.Co m 2. Click On Submit Button Without Entering Username and Password	A Validation Should Be As Below “Please Enter Valid Username & Password”	A Validation Has Been Populated As Expected	Pass	High

		1. aNavigate To Www.Sample.Co m 2. Click On Submit Button With Out Filling Password And With Valid Username	A Validation Should Be As Below “Please Enter Valid Password Or Password Field Can Not Be Empty “	A Validation Is Shown As Expected	Pass	High
		1. NNavigate To Www.Sample.Co m 2. Enter Both Username And Password Wrong And Hit Enter	A Validation Shown As Below “The Username Entered Is Wrong”	A Validation Is Shown As Expected	Pass	High
		1. Navigate To Www.Sample.Co m 2. Enter Validate Username And Password And Click On Submit	Validate Username And Password In DataBase And Once If They Correct Then Show The Main Page	Main Page/ Home Page Has Been Displayed	Pass	High

10.2 SCREENSHOTS

Increasing technology giving rise to errors and there is no automatic way to provide solution to generated error. Users always suffer from error and search solutions by sending queries to different servers and all servers will report some solutions and user has to identify correct solution from all those server responses which is very time consuming and difficult task.

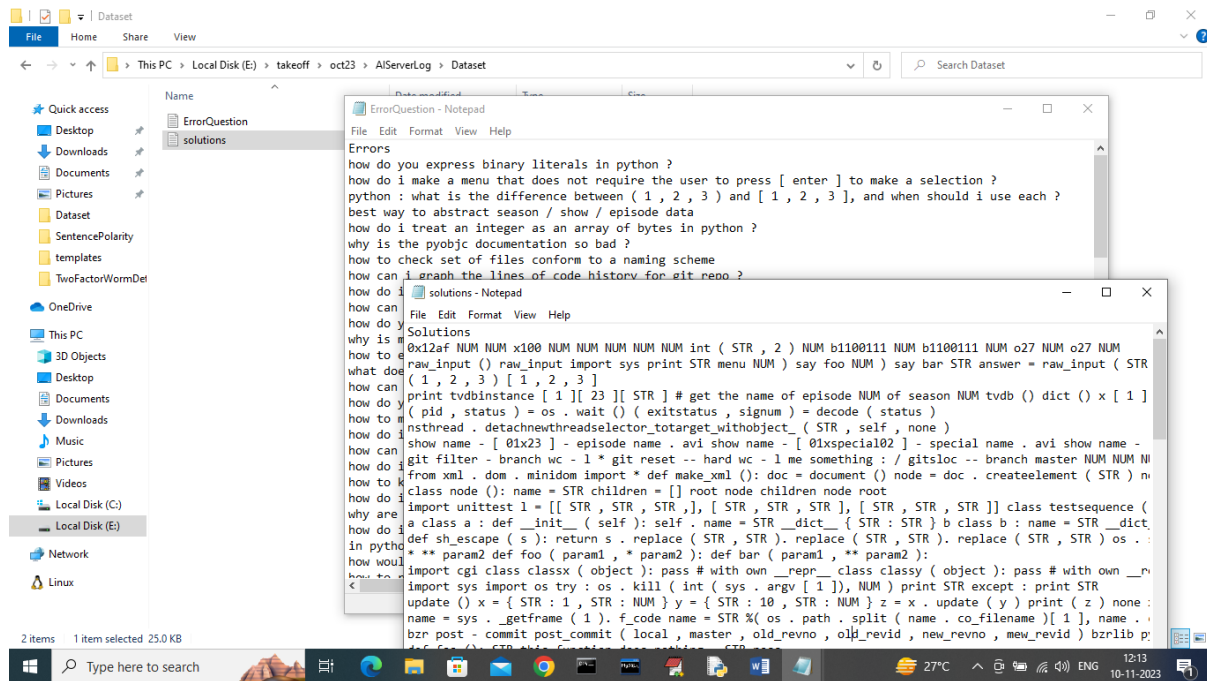
To provide easiest solutions we are obtaining all servers logs with errors and then providing solution to all those server errors and then training all those error questions and possible solutions with Artificial Intelligence algorithm. Trained AI model can take user test error as input and then predict possible close solution to given error. With this AI model user has to search and get solution from single model and can avoid searching from different servers which can save time and reduce user burden.

So by using above AI trained model user will get solution from single one stop server without searching in different servers

You ask to get solution from online websites like BING, Google and other servers but all this server may not give accurate result and Google will give 100's of results which is difficult to train and suggest close answer. So we have gather some possible questions and answers related to OS and programming and then use those questions and answers to train AI model.

If you want you can add new questions and answers to currently available dataset and you can add for other domains like OS errors, all programming errors and solutions and many more.

In below screen we are showing some errors/question and answers dataset details



In above dataset folder we have errors and solution dataset and by using above datasets will train and test AI algorithm

To implement this project we have designed following modules

Admin Module: admin can login to web application by using username and password as 'admin' and 'admin' and then train AI model and make it ready to take user errors and to provide solutions. Admin can view list of registered users to track his website usage

User Module: user can sign up and login to system and then enter his error details to get solution.

To clean all questions text we have used NLP (Natural language tool kit API) like stemming, lemmatization and stop word removal and special symbol removal to make solution prediction accurate.

To run project first we need to create database by copying content from DB.txt file and paste in MYSQL

SCREEN SHOT

To run project double click on 'run.bat' file to start python DJANGO server and get below output

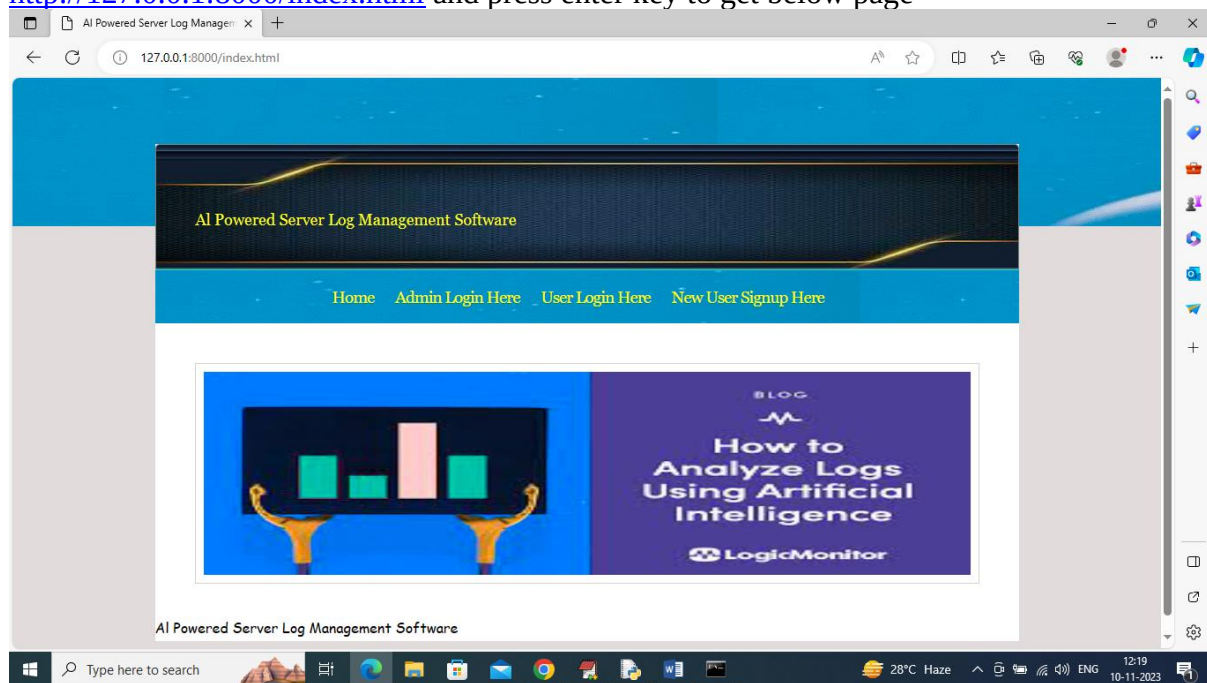
```
C:\Windows\system32\cmd.exe

E:\takeoff\oct23\AIServerLog>python manage.py runserver
C:\Users\Admin\AppData\Local\Programs\Python\Python37\lib\site-packages\pymysql\__init__.py
C:\Users\Admin\AppData\Local\Programs\Python\Python37\lib\site-packages\pymysql\__init__.py
Performing system checks...

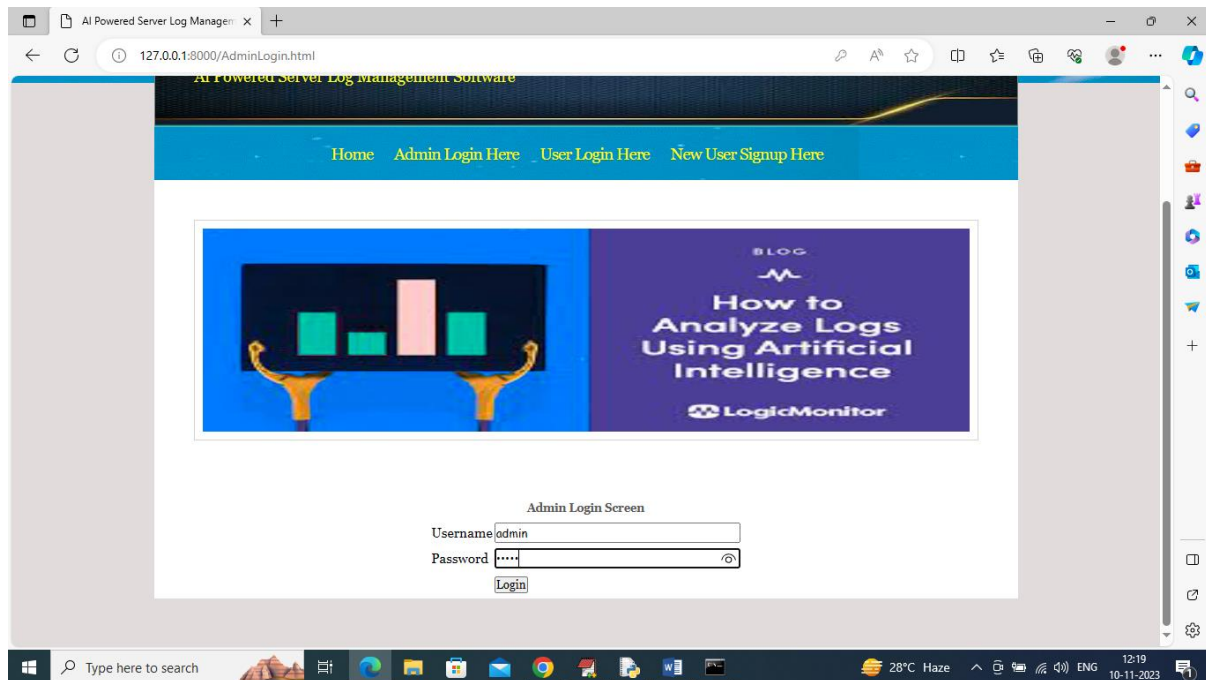
System check identified no issues (0 silenced).

You have 15 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin,
auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
November 10, 2023 - 12:18:46
Django version 2.1.7, using settings 'AILog.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

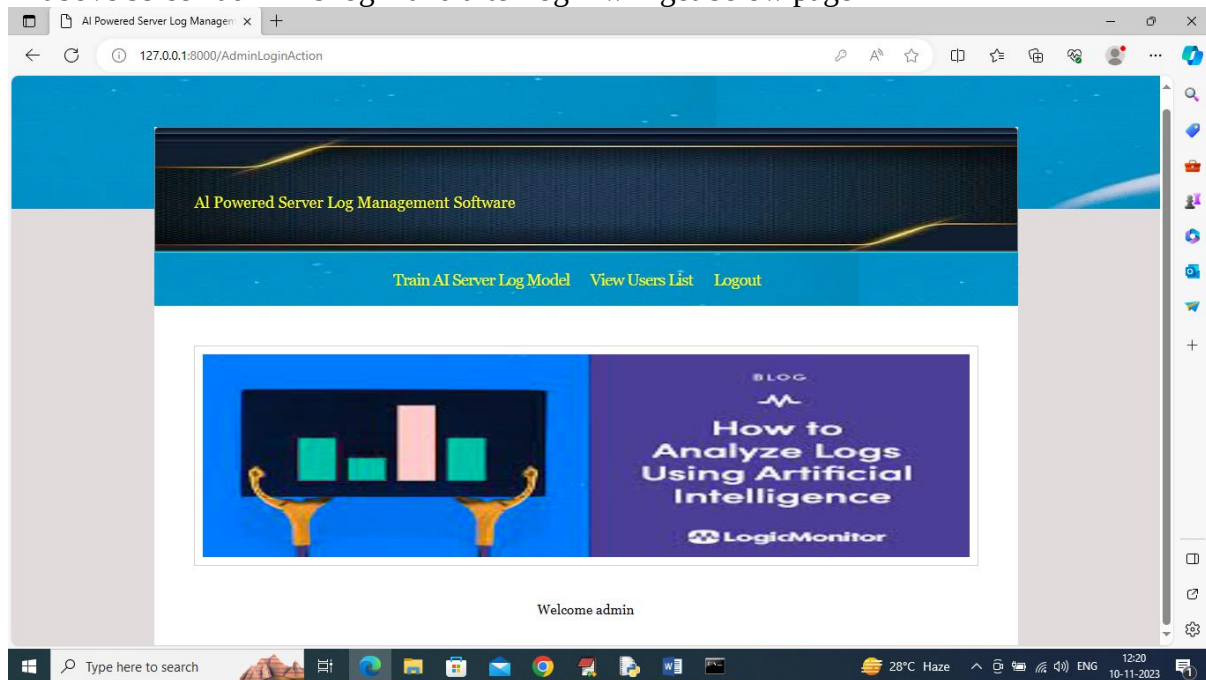
In above screen python server started and now open browser and enter URL as <http://127.0.0.1:8000/index.html> and press enter key to get below page



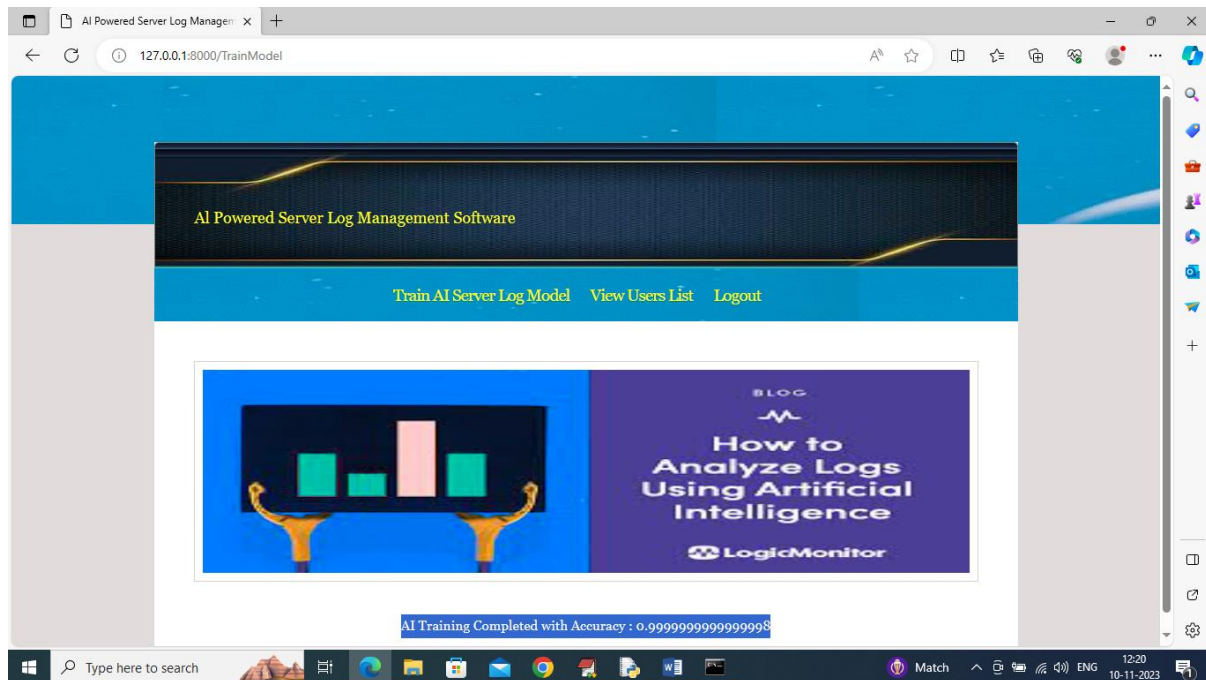
In above screen click on 'Admin Login Here' link to get below login page



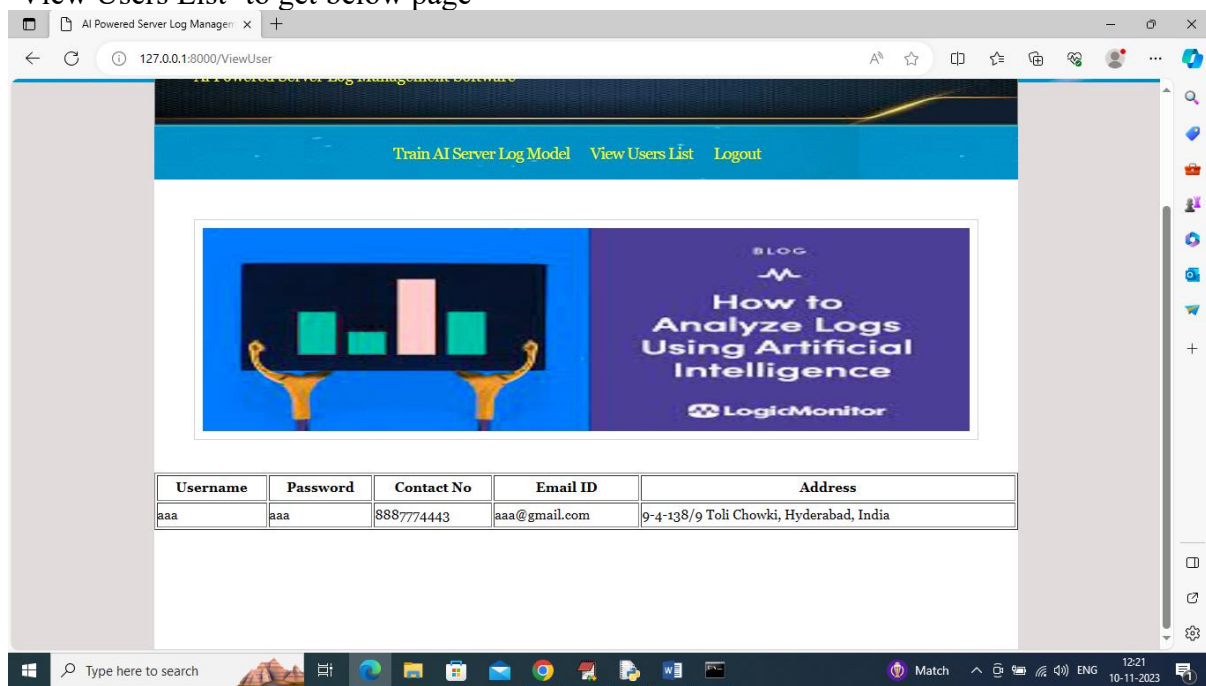
In above screen admin is login and after login will get below page



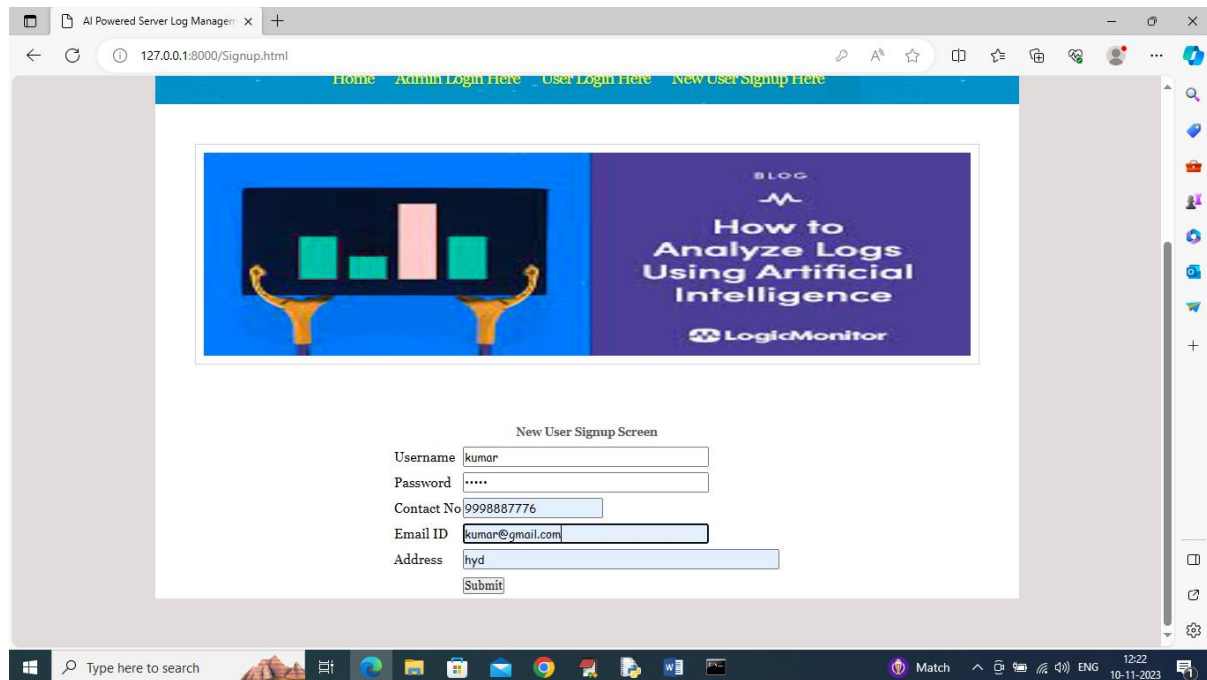
In above screen admin can click on 'Train Server Log Model' link to train AI model on dataset and then will get below output



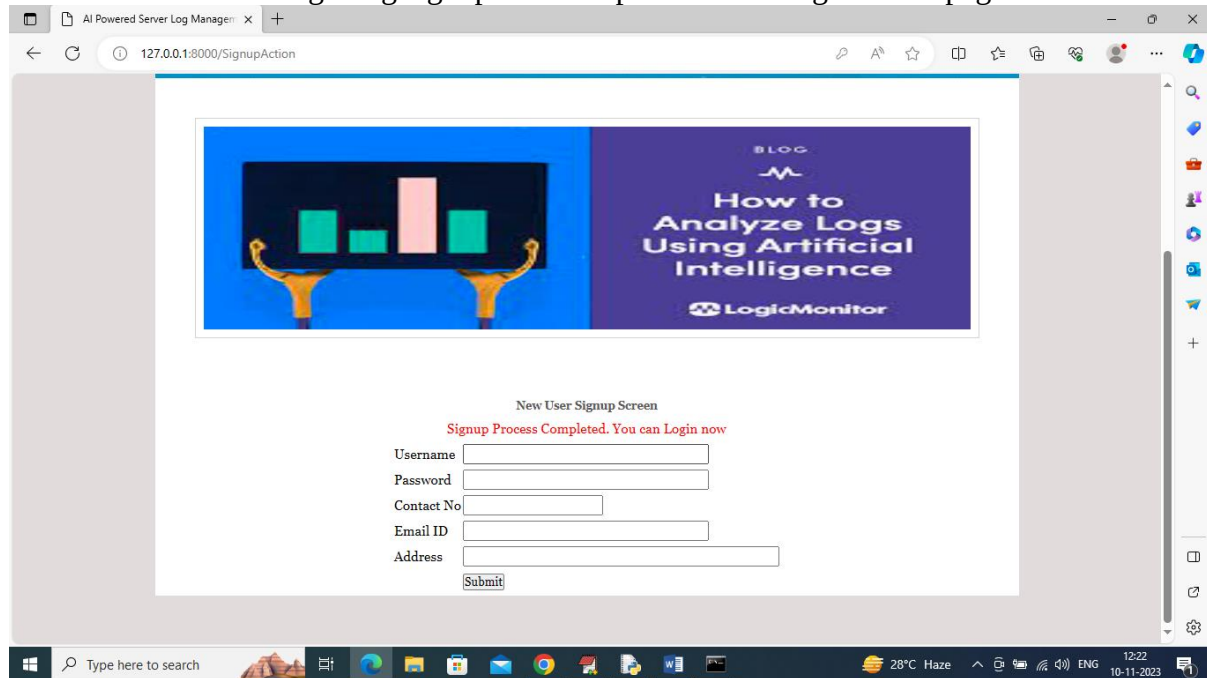
In above screen AI model training completed and its prediction accuracy is 0.99% and now click on 'View Users List' to get below page



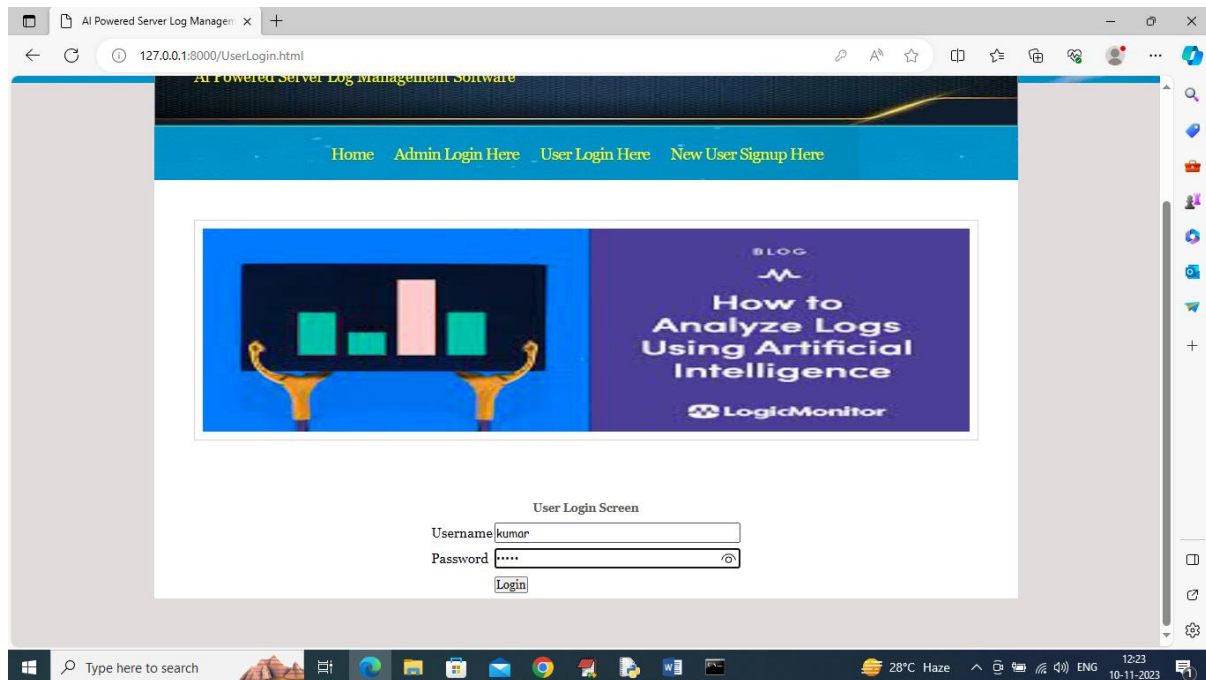
In above screen admin can view list of registered users and now logout and register one user to ask error from AI model



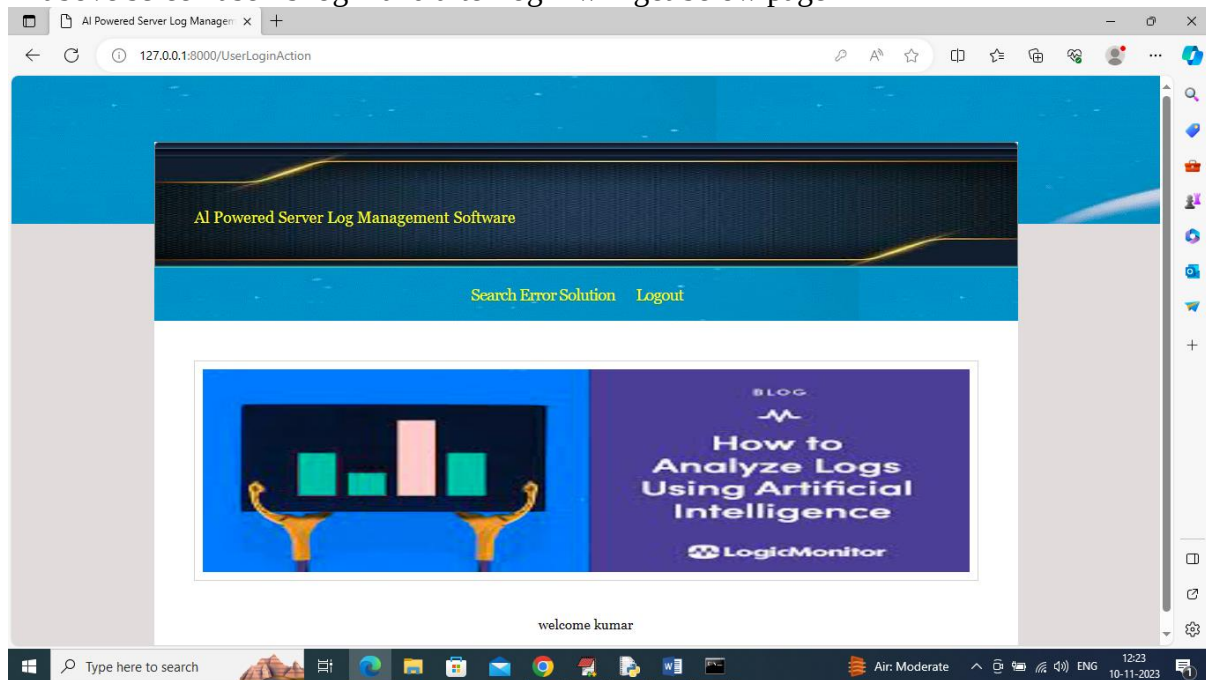
In above screen user is getting signup and then press button to get below page



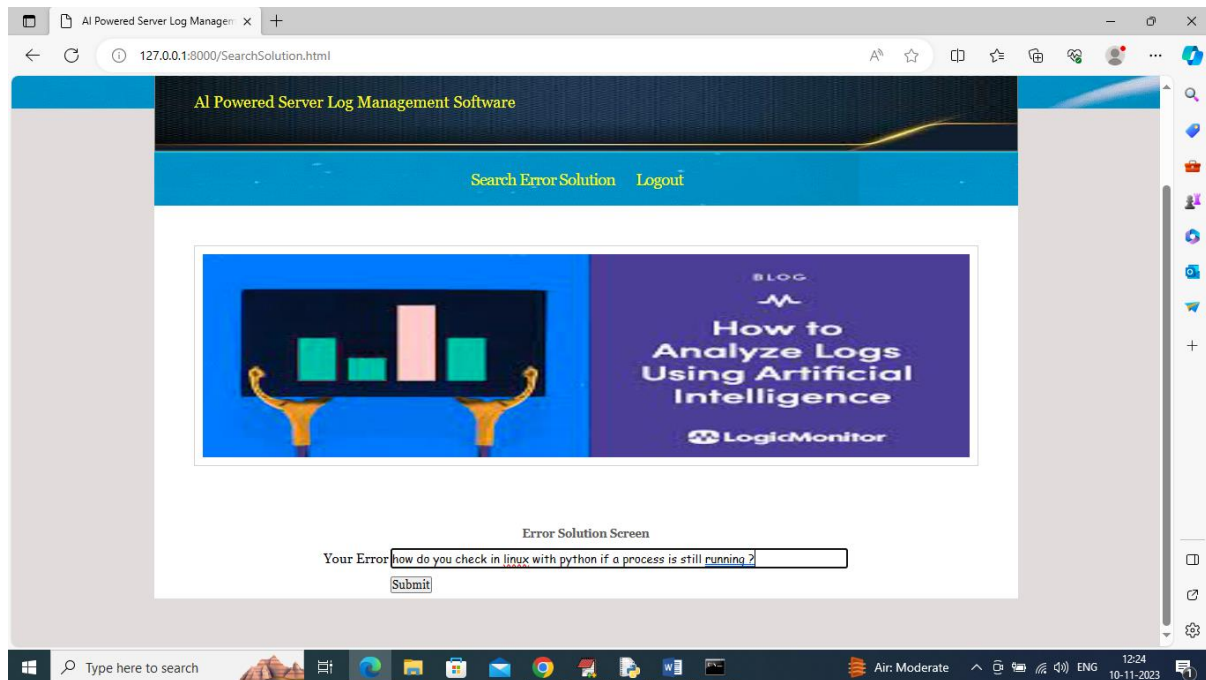
In above screen user signup completed and now click on 'User Login Here' link to get below page



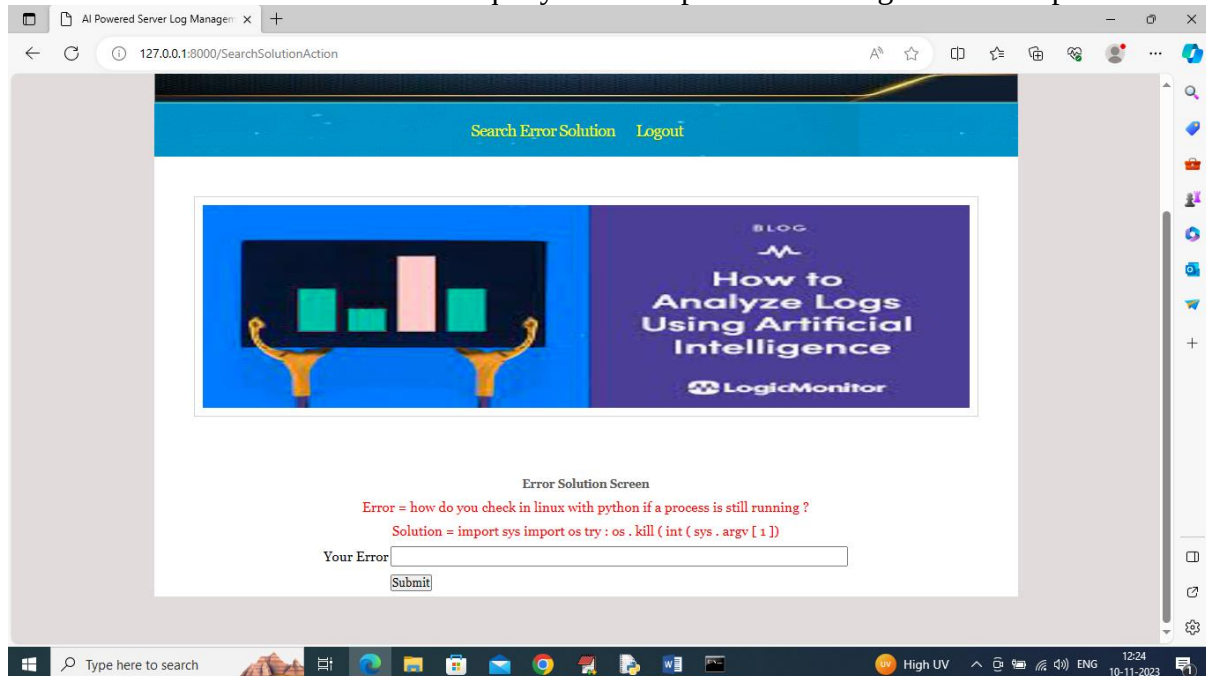
In above screen user is login and after login will get below page



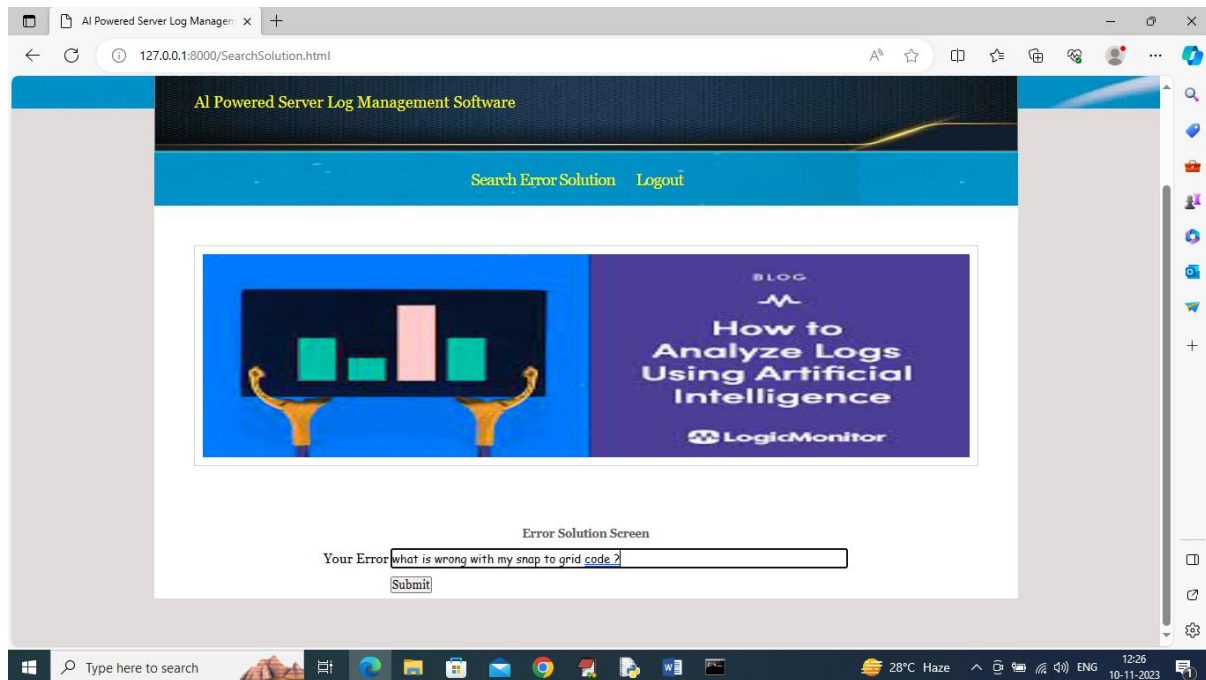
In above screen user can click on 'Search Error Solution' link to get below page



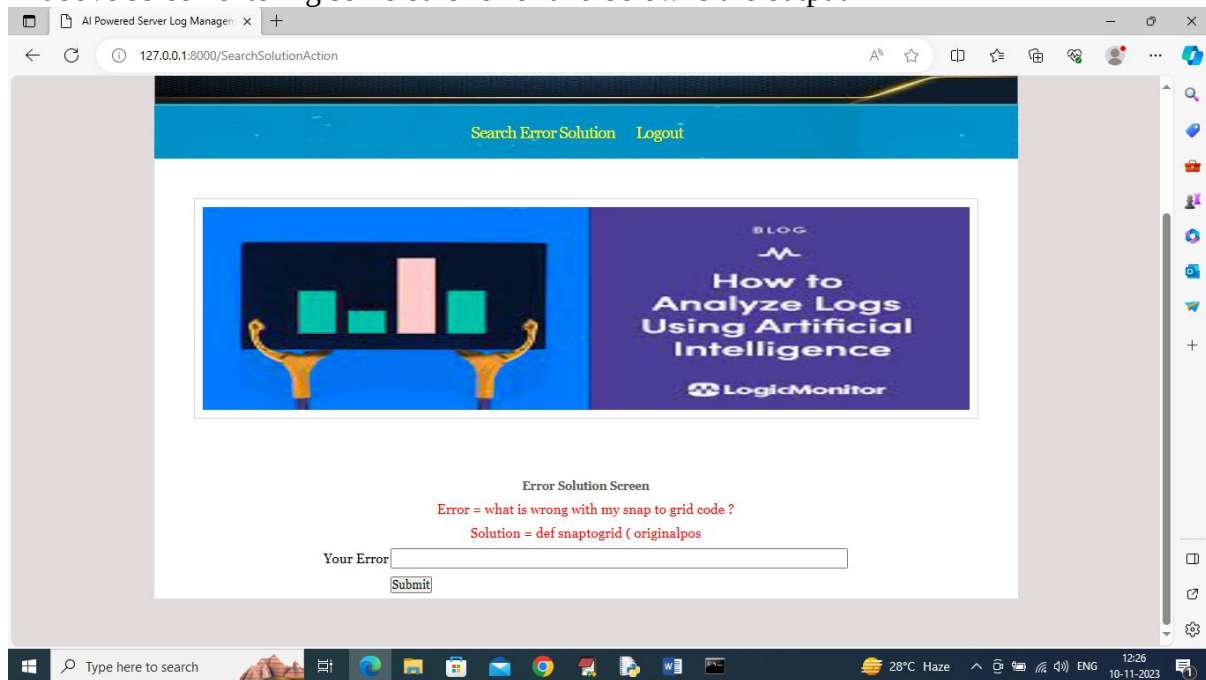
In above screen I entered some error query and then press button to get below output



In above screen in red colour text can see Error question and its possible solution and similarly you can enter any question and get output. If you want you can use some questions from 'testQuestions.txt' file which is available inside code folder and in below screen displaying another error output



In above screen entering some other error and below is the output



Similarly by following above screens you can ask any question from AI model to get solution

CHAPTER-11

CONCLUSION

11.1 CONCLUSION

In conclusion, the AI-powered Server Log Management Software project represents a paradigm shift in addressing the challenges associated with error resolution in server logs. By leveraging the capabilities of Artificial Intelligence, we are not only streamlining the process but also providing users with a centralized, efficient, and intelligent solution to their server-related issues. The proposed system marks a significant advancement in user experience, offering a more intuitive and responsive approach to managing server logs. As we continue to enhance and expand the dataset, incorporating new questions and answers, the system's accuracy and applicability across diverse domains will further contribute to its effectiveness in delivering timely and precise solutions.

11.2 FUTURE SCOPE

The future scope of AI-powered server log management software is promising, with several avenues for advancement and innovation. Here are some potential directions in which this technology may evolve:

1. Advanced Anomaly Detection:

Deep Learning Models: Continued advancements in deep learning techniques could lead to more accurate and efficient anomaly detection algorithms capable of identifying subtle patterns and anomalies in server logs.

Online Learning: Implementation of online learning algorithms that can continuously adapt to evolving data patterns and anomalies in real-time, ensuring proactive detection and mitigation of emerging threats.

2. Contextual Analysis:

Semantic Understanding: Integration of natural language processing (NLP) capabilities to extract context and semantics from log messages, enabling a deeper understanding of log events and their relevance to system behavior.

Domain-Specific Knowledge: Incorporation of domain-specific knowledge bases and ontologies to contextualize log data and improve the accuracy of anomaly detection by considering the unique characteristics of different environments.

3. Autonomous Response:

Automated Remediation: Development of AI-driven systems capable of autonomously responding to detected anomalies by implementing predefined remediation actions, such as adjusting system configurations, isolating compromised resources, or applying security patches.

Dynamic Policy Enforcement: Integration with policy-based management frameworks to dynamically enforce security policies and access controls based on real-time analysis of log data and threat intelligence feeds.

CHAPTER-12

REFERENCES

- [1] D. Yuan, S. Park, and Y. Zhou, “Characterizing logging practices in open source software,” in Proc. 34th Int. Conf. Softw. Eng. (ICSE), Jun. 2012.
- [2] M. Fuller, E. Brighton, M. Schiewe, D. Das, T. Cerny, and P. Tisnovsky, “Automated error log resolution: A case study,” in Proc. 36th Annu. ACM Symp. Appl. Comput., Mar. 2021.
- [3] X. Wang, D. Wang, Y. Zhang, L. Jin, and M. Song, “Unsupervised learning for log data analysis based on behavior and attribute features,” in Proc. Int. Conf. Artif. Intell. Comput. Sci., Jul. 2019.
- [4] R. C. Raga and J. D. Raga, “A comparison of college faculty and student class activity in an online earning environment using course log data,” in Proc. IEEE SmartWorld Ubiquitous Intell. Comput. Adv. Trust. Comput. Scalable, Dec. 2018.
- [5] G. Qi, W. T. Tsai, W. Li, Z. Zhu, and Y. Luo, “A cloud-based triage log analysis and recovery framework,” *Simul. Model. Pract. Theory*, vol. 77, Aug. 2020.
- [6] X. Tian, H. Li, and F. Liu, “Web service reliability test method based on log analysis,” in Proc. IEEE Int. Conf. Softw., Rel. Secur. Companion (QRS-C), Jul. 2017.
- [7] D. Zou, H. Qin, and H. Jin, “UiLog: Improving log-based fault diagnosis by log analysis,” *J. Comput. Sci. Technol.*, vol. 31, no. 5, 2016, doi: 10.1007/s11390-016-1678-7.
- [8] S. Locke, H. Li, T. H. P. Chen, W. Shang, and W. Liu, “LogAssist: Assisting log analysis through log summarization,” *IEEE Trans. Softw. Eng.*, early access, May 26, 2021, doi: 10.1109/TSE.2021.3083715.
- [9] J. Cândido, M. Aniche, and A. Van Deursen, “Log-based software monitoring: A systematic mapping study,” *PeerJ Comput. Sci.*, vol. 7, Oct. 2021.
- [10] D. Obrąbski and J. Sosnowski, “Log based analysis of software application operation,” *Adv. Intell. Syst. Comput.*, vol. 987, Oct. 2020 .
- [11] Q. Cao, Y. Qiao, and Z. Lyu, “Machine learning to detect anomalies in web log analysis,” in Proc. 3rd IEEE Int. Conf. Comput. Commun. (ICCC), Dec. 2017.

[12] B. Debnath, M. Solaimani, M. A. G. Gulzar, N. Arora, C. Lumezanu, J. Xu, B. Zong, H. Zhang, G. Jiang, and L. Khan, “LogLens: A real-time log analysis system,” in Proc. IEEE 38th Int. Conf. Distrib. Comput. Syst.(ICDS), July 2018.